
Java/Spring 환경의 동시성 문제 탐지 및 제어 기법별 성능 분석

Race Condition을 증명하고, 4가지 제어 기법의 성능 트레이드오프를 나노초 단위로 시각화

목차

1. 동시성 시나리오 테스트 목적
 - 1.1 프로젝트 개요 및 실험 목적
 - 1.2 프로젝트 중점 사항
 - 1.3 프로젝트 구조
2. 동시성 테스트 분석 기준 및 환경 설정
 - 2.1 공통 계측·분석 기준
 - 2.1.1 계측 항목 및 집계 지표
 - 2.1.2 계측 방식 및 자동화 집계
 - 2.1.3 계측·분석 데이터 구조
 - 2.1.4 계측·분석 항목 표
 - 2.2 계측 환경
3. 동시성 시나리오 테스트 정의 및 계측 포인트
4. 집계 방법
5. 동시성 시나리오 테스트 및 결과 분석
 - 5.1 테스트 환경 정의
 - 5.1.1 테스트 요구 사항 정리
 - 5.1.2 실험 실행 조건
 - 5.1.3 계측·검증 항목
 - 5.1.4 실험 자동화 및 계측 도구
 - 5.2 계측 데이터
 - 5.2.1 전체 요약 보고서
 - 5.2.2 조건 분기 (IF-ELSE)
 - 5.2.3 모니터 락(synchronized)
 - 5.2.4 ReentrantLock
 - 5.2.5 세마포어(Semaphore)
 - 5.3 동시성 제어 구조별 종합 성능·상태 비교 분석

동시성 시나리오 테스트 목적

1.1 프로젝트 개요 및 실험 목적

본 보고서는 실무 시스템에서 자원 점유 통제를 설계함에 있어 동기화 구조의 통제 단위가 정합성과 병목 회피를 동시에 충족할 수 있는가라는 구조적 물음을 출발점으로 한다. 이 실험은 성능 비교가 아닌 통제 구조의 설계 타당성에 대한 이론적 검증을 핵심 목표로 한다.

조건 분기 기반 구조는 상태 판단과 자원 갱신이 분리되어 있어 동시 접근 시 판단 이후 상태가 변경되는 경우를 제어할 수 없다. 이는 구조적으로 race condition을 방지할 수 없음을 의미하며, 병목은 발생하지 않지만 결과는 항상 비일관성이다.

뮤텍스 락 기반 구조는 조건 판단과 상태 갱신을 임계 구역 내에서 수행하므로 자원 보호는 가능하지만, 모든 진입자가 단일 락을 획득해야 하므로, 동시 요청이 집중될 경우 지연이 누적되며 병목이 발생한다. 또한 락 내부에서 상태 객체가 외부에 의존하면 일관성 유지가 구조 외부에 의존하게 된다.

세마포어 기반 구조는 점유 허용량을 permit 단위로 명시적으로 표현하며, tryAcquire 시점에서 판단과 자원 점유가 동시에 atomic하게 이루어진다. 이는 병목을 피하고 race를 차단하며, 통제 단위가 구조 내부에 응집된다는 점에서 설계상 가장 명료한 통제 경계를 가진다.

이러한 의문은 단순 성능 비교나 구현 편의성의 문제가 아닌 설계 구조 자체의 정당성을 판단하기 위한 것이다. 따라서 본 보고서는 관측된 문제에 대한 분석이 아니라, 설계적 의심을 실험적으로 입증하기 위한 목적에서 출발한다.

1.2 프로젝트 중점 사항

본 보고서는 실시간 서비스 시스템의 동시 입장 제어 구조가 단순한 기능적 구현의 효율성이나 처리량 비교에 그치는 것이 아니라, 각 구조가 실제 운영 환경에서 race condition의 구조적 차단, 병목의 실질적 억제, 상태 일관성의 정량적 입증, 그리고 반복 실험을 통한 결과의 일관성과 재현성까지 통합적으로 검증할 수 있는지에 중점을 두고 설계되었다. 모든 계측 기준과 실험 절차는 개별 사례의 성공/실패가 아닌, 동일 조건에서 반복 수행 시 일관된 결과가 재현되는지, 그리고 각 구조가 내포한 설계적 한계가 실제 데이터상 어떻게 표출되는지를 명확히 드러내는 데 집중된다.

따라서, 실험의 모든 평가 관점은 단일 스냅샷 기반의 일시적 수치가 아닌, race condition 발생 구간의 실시간 검출, 그리고 대량 동시 요청 환경에서의 병목 및 경합 발생 시점의 자동 검출로 귀결된다.

이러한 실험적 중점 사항은 단순히 구조별 성능 차이를 논증하는 데 그치지 않고, 실제 서비스 수준에서 요구되는 구조적 안정성과 실무적 신뢰성을 객관적 데이터로 입증하는 데 있으며, 궁극적으로는 설계 선택의 명확한 기준, 실무 적용성 평가, 실험 결과의 재현성과 검증 가능성을 최우선 원칙으로 한다.

1.3 프로젝트 구조

본 보고서는 실험의 계측·분석 원칙을 먼저 고정한 뒤, 각 구조별 실험 시나리오의 전개와 결과 해석, 실무적 타당성 평가까지의 전 과정을 계층적으로 기술하였다.

- 계측·분석 기준과 실험 자동화 환경을 명확히 정의하여, 실험의 객관성과 반복 가능성, 데이터 신뢰성을 확보하는 방법론을 기술한다.
- 각 동시성 제어 구조별 실험 시나리오 및 계측 포인트를 구체적으로 서술하여, 구조별 차별화된 측정 항목과 분석 프레임을 제시한다.
- 실험 결과의 정량적 데이터 및 주요 현상을 비교 분석하여, 구조적 병목, race condition 발생 여부, 상태 일관성 붕괴 및 복구 가능성 등 실질적 관찰 데이터를 통계적으로 제시한다.
- 각 구조별 실험 결과의 해석과 실무적 효과, 한계 및 확장 방향을 심층적으로 분석하며, 단순 수치 비교를 넘어 구조별 설계 선택의 실무 적용성과 장기적 확장성까지 입증한다.
- 실험 결과에 기반한 종합 결론 및 구조 선택 가이드를 제시하여, 실무 환경에서의 최적 선택 기준, 적용 시 주의점, 향후 연구 및 개선 방향 등을 체계적으로 정리한다.

이러한 전체 구조는 실험 설계의 철학적 배경과 계측의 객관성, 결과의 재현성과 실무 적용성까지 일관된 논리로 연결되는 것을 목표로 하며, 각 장의 배치와 내용은 모든 실험적 관찰이 단순 현상 나열이 아니라 구조적 타당성과 실증적 입증으로 이어지도록 설계되었다.

동시성 테스트 분석 기준 및 환경 설정

공통 계측·분석 기준

구분	계측 항목	정의	집계·측정 방식	분석 및 활용 관점
데이터 집계	처리량 (ops/sec)	단위 시간 내 처리 완료 건수	자동 스크립트·JMeter 통계 집계	구조별 성능 비교, 시스템 최대 처리량 검증
	응답 시간	요청~응답 완료까지 전체 소요 시간	JMeter/클라이언트 자동 로그	사용자 체감 성능, End-to-End 성능 변화
	대기 시간	임계구역 진입 대기 구간 소요 시간	서버 로그/스크립트 계측	경합/락 경쟁 병목 검증, 구조별 차이
	임계구역 처리 시간	임계구역 점유~처리 완료까지 소요 시간	서버 로그/스크립트 계측	락 점유/병목 구간, 구조적 효율성 평가
이벤트·상태 집계	race condition	허용 인원 초과/상태 불일치 등 구조적으로 문제 발생 건수/비율	상태 snapshot, 자동 로그	구조적 결함, 동시성 오류, race 예방 효과
반복성/재현성	10회 반복 통계 집계	각 구조별·조건별 10회 이상 반복·통계적 신뢰성 확보	자동 반복, 독립 로그 파일 저장	통계적 유의성, 실험 신뢰도
시각화	분포·변동 분석	표, 히스토그램, boxplot 등 다양한 시각화	matplotlib, pandas, Excel 등	구조별 처리량/병목/이상값 등 시계열·분포 분석
결과 검증/증적	증적 파일/로그	실험 결과/스냅샷/자동 로그 등 결과물 증적화	자동 파일·로그 저장, snapshot	결과 재현, 보고서·포트폴리오 근거 제공

동시성 시나리오 테스트 정의 및 계측 포인트

본 장에서는 2장에서 정의한 공통 계측/분석 기준을 전제로, 각 동기화 구조(조건 분기/무텍스 락/세마포어/리엔트런트락)의 계측 시나리오와 구조적 차별화 포인트, 그리고 계측 방식(ops/sec, 병목, 상태 일관성 등) 및 결과 해석의 실질적 기준을 명확히 제시한다.

모든 시나리오는 처리량(ops/sec), 응답 지연, 상태 일관성, race condition, 반복 실험, 데이터 분석 등 2장 기준을 따른다.

동시성 테스트 시나리오

1. 조건 분기 기반 구조 (If-Else)

[실험 시나리오 및 계측 플로우]

- Step 1: 200명 동시 입장 요청(자동화 도구 이용), 각 요청은 currentCount < MAX 조건 분기 후 입장 시도
- Step 2: 조건 통과 시 상태 객체(Map/Counter) 직접 갱신, 입장 성공 처리
- Step 3: 모든 요청의 상태 로그를 자동 저장
- Step 4: 입장 성공·실패·중복 등 이벤트 발생 즉시 자동 로그 및 이벤트 카운트
- Step 5: 10회 반복, 평균/최대/최소/표준편차 집계

2. 무텍스 락 기반 구조 (Mutex/Synchronized)

[실험 시나리오 및 계측 플로우]

- Step 1: 200명 동시 입장 요청, 각 요청은 synchronized/lock 임계구역 진입 시도
- Step 2: 임계구역 내에서 currentCount < MAX 판단 및 상태 객체(Map/Counter) 갱신
- Step 3: 임계구역 진입 대기, 점유, 해제 시각 자동 계측(락 경쟁, 대기 시간 포함)

3. ReentrantLock 기반 구조

[실험 시나리오 및 계측 플로우]

- Step 1: 200명 동시 입장 요청, 각 요청은 ReentrantLock.lock()으로 임계구역 진입 시도
- Step 2: 임계구역 내 currentCount < MAX 판별 및 상태(Map/Counter) 갱신
- Step 3: 락 진입 대기/점유/해제/반납 시각, 대기 큐 길이, 락 획득 대기 시간 등 자동 계측
- Step 5: 이벤트 로그 자동 저장, 10회 반복 통계 집계

4. 세마포어 기반 구조 (Semaphore)

[실험 시나리오 및 계측 플로우]

- Step 1: 200명 동시 입장 요청, 각 요청은 Semaphore.tryAcquire()로 permit 원자적 점유 시도
- Step 2: permit 점유 성공 시 상태 객체(Map/Counter/permit) 동기화, 실패 시 거부 처리
- Step 3: permit/세션/DB 상태 snapshot 및 모든 이벤트(중복, 초과, 불일치 등) 자동 저장
- Step 4: tryAcquire/release 타이밍 및 경쟁 상황 자동 로그
- Step 5: 10회 반복, 통계 집계

동시성 테스트 별 관찰 기준

동기화 구조 유형	계측 항목 (세부 관찰/실험 포인트)	측정/관찰 방식 (정량/정성)	해석/분석 관점 (실험 목적)
조건 분기 기반	동시 입장 중복 진입률 / race condition 빈도	이벤트 카운트, WARN/Error 로그, 자동 계측	조건-실행 분리의 구조적 한계, race condition 불가피 발생, 실무 적용 한계 증거화
	상태 일관성(불일치율)	이벤트 로그, race condition 유무	반복적 불일치/장애(중복 입장, 인원 초과, 데이터 불일치) 실증
무텍스 락 기반	race condition 차단율	이벤트 로그, race 유무 자동 계측	100% 근접 시 동시성 보호 효과, 구조적 race 차단 효과 입증
	병목/응답 지연	평균/최대 응답 시간, 처리량(ops/sec)	락 경쟁에 따른 처리량 저하/확장성 한계 실증
	상태 일관성/결함	상태 스냅샷, 이벤트 로그	락 내부/외부 상태 경계/제한 명확화, 동기화 범위 한계
세마포어 기반	race condition 발생률	이벤트 로그, race condition 유무	0% 일치 시 CAS 기반 atomic 구조의 오류 차단 실증
	처리량/응답 시간	처리량(ops/sec), 평균/최대 응답 시간	락 기반 대비 우수, 병목 원천 차단, 구조적 효과 입증
	상태 일관성	permit/세션/DB 상태 스냅샷, 자동 비교	반복적 유지 시 permit 기반 활동 통제 구조 신뢰성 수치로 증명

동시성 시나리오 테스트 및 결과 분석

테스트 환경 정의

계측 데이터

동시성 제어 구조별
종합 성능·상태 비교 분석

테스트 환경 정의

1. 테스트 환경

구분	도구/프레임워크	용도 및 목적
부하 시뮬레이션	JMeter	동시 입장/요청/퇴장 등 대량 시나리오 자동 반복 및 응답 계측
서버 로그 계측	Log4j2 (Spring Boot)	race 발생 등 주요 이벤트 자동 로깅
상태/이벤트 캡처	Shell, Python Script	실험 반복별 상태 dump, 복구 이벤트 발생 시점 캡처 자동화
데이터 저장/분석	Excel, CSV, Pandas	반복 계측 데이터 저장, 통계 분석, 정량 결과 산출
시각화	matplotlib, Excel Chart	처리량/응답시간/에러/복구 시계열 및 분포 그래프 생성

2. 계측 지표 정의 및 집계 기준

구분	계측 지표	정의/설명	집계·측정 방식	분석/활용 관점
서버 측	race condition 발생률	실험 중 상태 불일치, 중복 갱신 등 race 이벤트 비율	서버 로그/자동 스크립트	동시성 구조 결함 검증 및 경합 예방 효과 검증
	정원 초과 진입 비율 (over-capacity)	최대 인원 초과 시도 건수 및 전체 대비 비율	서버 로그/자동 스크립트	구조별 경합/병목 현상 계량, 정책 효과 검증
	대기 시간 (waitingMillis)	임계구역 진입 대기 구간 소요 시간	서버 로그/스크립트 통계	구조별 경합/락 병목 분석, 동시성 개선 효과
	임계구역 처리 시간 (criticalElapsed)	임계구역 점유~처리 완료까지 소요 시간	서버 로그/스크립트 통계	락 점유/병목 구간 분석, 구조적 효율성 평가
클라이언트 측	응답 시간 (Response Time)	클라이언트 요청~응답 완료까지 전체 소요 시간 (평균/중앙값/p99 등)	JMeter 등 부하 시뮬레이터 자동 집계	End-to-End 성능, 사용자 체감 품질 변화
	처리량 (Throughput, ops/sec)	단위 시간(초)당 성공적으로 처리된 요청 건수	JMeter 등 부하 시뮬레이터 자동 집계	시스템 성능 한계 및 구조별 최대 처리량 비교

동시성 시나리오 테스트 및 결과 분석

전체 요약 보고서 - 서버 지표 : RaceCondtion 지표

본 절의 목적은 다중 스레드 요청 환경에서 서버의 각기 다른 동기화 구조(조건 분기, 뮤텝스 락, 세마포어)가 데이터 정합성과 시스템 안정성을 어떻게 보장하는지 실증적으로 분석하는 것이다. 10회 반복 계측을 통해 각 구조의 Race Condition 발생률, 성능 지표로써 이론이 아닌 실측 데이터에 기반하여, 어떤 아키텍처가 서버의 내부 상태를 가장 안정적으로 유지하고 시스템 부하를 효율적으로 처리하는지 증명한다.

[각 계측 항목 별 10회 테스트 수행 결과 표 - RaceCondtion 규칙 1 : 값 불일치]

계측 항목	조건 분기	Synchronized	ReentrantLock	세마포어 기반
전체 요청 수	2000	2000	2000	2000
값 불일치 발생 횟수 / 비율(%)	127 / 6.35 %	0 / 0 %	0 / 0 %	-
값 불일치 누적 총합 값	168	0	0	-
값 불일치 누적 최대 값	5	0	0	-
값 불일치 누적 최소 값	1	0	0	-
값 불일치 누적 평균 값	1.32	0	0	-

[각 계측 항목 별 10회 테스트 수행 결과 표 - RaceCondtion 규칙 2 : 경합 발생]

계측 항목	조건 분기	Synchronized	ReentrantLock	세마포어 기반
전체 요청 수	2000	2000	2000	2000
경합 발생 스레드 수 / 발생률(%)	1839 / 91.95	0 / 0 %	0 / 0 %	974 / 48.7 %
경합 발생 총합 값	9618	0	0	-
경합 발생 최대 값	18	0	0	-
경합 발생 최소 값	2	0	0	-
경합 발생 평균 값	5.23	0	0	-

[각 계측 항목 별 10회 테스트 수행 결과 표 - RaceCondtion 규칙 3 : 정원 초과 오류]

계측 항목	조건 분기	Synchronized	ReentrantLock	세마포어 기반
전체 요청 수	2000	2000	2000	2000
정원 초과 입장 건수 / 비율 (%)	1 / 0.05	0 / 0 %	0 / 0 %	0 / 0 %
정원 초과 최대 값	1	0	0	0
정원 초과 최소 값	1	0	0	0
정원 초과 평균 값	1	0	0	0

[각 계측 항목 별 10회 테스트 수행 결과 표 - RaceCondtion 규칙 4 : 상태 전이 오류]

계측 항목	조건 분기	Synchronized12816.1	ReentrantLock	세마포어 기반
전체 요청 수	2000	2000	2000	2000
상태 전이 발생 횟수 / 비율(%)	183 / 9.15	0 / 0 %	0 / 0 %	0 / 0 %
상태 전이 누적 총 값	242	0	0	-
상태 전이 누적 최대 값	4	0	0	-
상태 전이 누적 최소 값	-3	0	0	-
상태 전이 누적 평균 값	1.32	0	0	-

동시성 시나리오 테스트 및 결과 분석

전체 요약 보고서 - 서버 지표 : 임계구역 대기 시간과 임계구역 수행 시간

본 절의 목적은 다중 스레드 요청 환경에서 서버의 각기 다른 동기화 구조(조건 분기, 뮤텝스 락, 세마포어)가 데이터 정합성과 시스템 안정성을 어떻게 보장하는지 실증적으로 분석하는 것이다. 10회 반복 계측을 통해 각 구조 별 서버의 핵심 신뢰성 지표를 정량적으로 평가한다. 이를 통해 이론이 아닌 실측 데이터에 기반하여, 어떤 아키텍처가 서버의 내부 상태를 가장 안정적으로 유지하고 시스템 부하를 효율적으로 처리하는지 증명하여, 견고한 백엔드 설계를 위한 기술적 타당성을 확보한다.

[각 계측 항목 별 10회 테스트 수행 결과 표]

계측 항목	조건 분기 기반 (Non Validate)	조건 분기 기반 (Validate)	Synchronized	Synchronized (Double-Checked)	ReentrantLock (non_Faire)	ReentrantLock (Faire)	세마포어 기반
전체 요청 수	2000		2000	2000	2000	2000	2000
방 입장 성공 수 / 비율 (%)	991 / 49.55		990 / 49.50	990 / 49.50	990 / 49.50	990 / 49.50	990 / 49.50
방 입장 실패 수 / 비율 (%)	1009 / 50.45		1010 / 50.50	16 / 0.8	1010 / 50.5	1010 / 50.5	1010 / 50.5
임계구역 진입 실패 수 / 비율 (%)	0 / 0.00		0 / 0.00	0 / 0.00	0 / 0.00	0 / 0.00	0 / 0.00
방 입장 성공 대기시간 평균 (μs)	48.4		353.09	203.73	2109.5	972.38	-
방 입장 성공 대기시간 최대 (μs)	1,609.1		44233.7	39975.5	164034.8	102282.2	-
방 입장 성공 대기시간 중간 (μs)	22.0		25.2	26.15	52.4	52.4	-
방 입장 실패 대기시간 평균 (μs)	83.2		631.28	2897.33	4094.8	4074.6	-
방 입장 실패 대기시간 최대 (μs)	7,061.0		12816.1	7887.3	25784	29803.5	-
방 입장 실패 대기시간 중간 (μs)	20.8		53350	2695.05	2385.85	2096.6	-
방 입장 성공 임계구간 실행 시간 평균 (μs)	140.6		133.26	111.88	355.19	259.33	413.06
방 입장 성공 임계구간 실행 시간 최대 (μs)	4,073.0		35088.3	14577.5	50479	102301.9	38176.7
방 입장 성공 임계구간 실행 시간 중간 (μs)	54.1		51.5	52.65	82.15	81.85	86.15
방 입장 실패 임계구간 실행 시간 평균 (μs)	145.6		65.1935	122.91	166.72	106.16	971.01
방 입장 실패 임계구간 실행 시간 최대 (μs)	8,148.7		896.6	734	16773.9	1301	23607.1
방 입장 실패 임계구간 실행 시간 중간 (μs)	26.8		33.3	50.4	65.55	65.55	119.15

전체 요약 보고서 - 클라이언트 지표

본 절의 목적은 서버의 다양한 동시성 제어 구조(조건 분기, 뮤텝스 락, 세마포어)가 클라이언트의 사용자 경험에 어떤 영향을 미치는지 실증적으로 측정하는 것이다. 10회 반복 테스트를 통해 클라이언트가 체감하는 응답 시간(Latency), 처리량(Throughput), 요청 성공률 등 핵심 성능 지표를 집계·분석한다. 이를 통해 단순 서버의 내부 로직을 넘어, 어떤 백엔드 구조가 최종 사용자에게 가장 빠르고 안정적인 서비스를 제공하는지 데이터에 기반하여 증명하고, 최적의 사용자 경험을 위한 아키텍처 선택의 근거를 마련한다.

[각 계측 항목 별 10회 테스트 수행 결과 표 - 클라이언트 지표 : Jmeter 테스트]

계측 항목	조건 분기 기반	Synchronized	ReentrantLock (non_Fair)	ReentrantLock (Faire)	세마포어 기반
표본 수	2000	2000	2000	2000	2000
평균 값 (ms)	200	168	198	152	231
중간 값 (ms)	265	157	154	147	209
99% 선 (ms)	305	403	903	332	442
최소 값 (ms)	75	9	31	21	28
최대 값 (ms)	306	427	993	344	462
오류 %	0.00	0.00	0.00	0.00	0.00

[JMeter 핵심 지표별 의미]

- 표본 수 : 특정 요청을 서버에 보낸 총 횟수.
- 평균 : 모든 요청의 응답 시간을 산술 평균한 값으로, 전체적인 성능 수준.
- 중앙값 : 응답 시간을 순서대로 나열했을 때 중앙에 위치하는 값으로 극단적인 이상치(outlier)의 영향을 덜 받아 일반적인 사용자의 경험을 더 잘 반영함.
- 99% : 각각 전체 요청의 99% 이 시간 안에 처리되었음을 의미함.
- 최소/최대 : 기록된 가장 빠른 응답 시간과 가장 느린 응답 시간.
- 오류 % : 전체 요청 중 실패한 요청의 비율로, 서비스의 안정성을 나타내는 직접적인 지표.

동시성 시나리오 테스트 및 결과 분석

'Check-Then-Act' 구조의 동시성 결함 실증 - RaceCondtion 규칙 1 : 값 불일치

조건 분기(If-Else) 구조는 동시성 환경에서 데이터 정합성을 보장할 수 없는 대표적인 안티패턴이다. 본 절의 목적은 이 'Check-Then-Act' 로직이 실제 부하 상황에서 어떻게, 그리고 왜 실패하는지를 정량적 데이터로 증명하는 데 있으며, 이를 위해 해당 로직에 JMeter로 동시 요청 부하를 가하는 실험을 총 10회 반복 수행했으며, 이 과정에서 발생한 모든 상태 변화를 나노초 단위의 로그로 기록하였다.

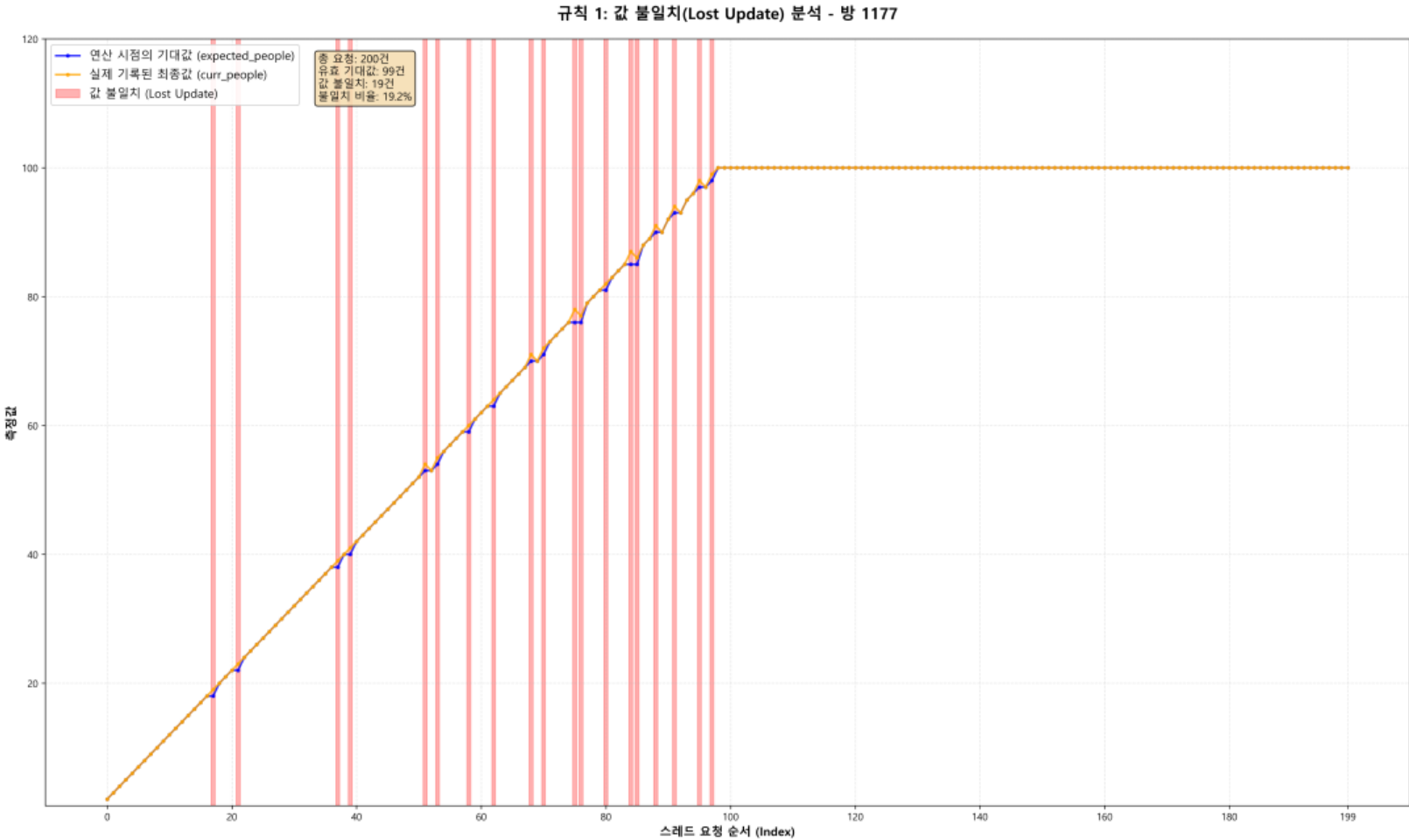
아래에 제시된 데이터는 각 실험 회차에서 관측된 데이터를 통해 '값 불일치(Lost Update) 오류가 어떤 패턴과 빈도로 발생하는지, 그리고 왜 실무에서 명시적인 락(Lock) 매커니즘이 반드시 필요한지를 명확히 확인할 수 있다.

[표 – 계측 데이터]

방 번호	bin	순번	사용자 ID	이전 인원	기대인원	현재인원	오차
1177	1	18	yhjj852	17	18	19	1
1177	2	22	yhjj977	21	22	23	1
1177	4	76	yhjj847	75	76	78	2
1177	4	77	yhjj889	75	76	77	1
1177	5	81	yhjj998	80	81	82	1
1177	5	85	yhjj853	84	85	87	2
1177	5	86	yhjj999	84	85	86	1
1177	5	89	yhjj1009	89	90	91	1
1177	5	92	yhjj925	92	93	94	1
1177	5	96	yhjj870	96	97	98	1

[데이터 분석]

[차트]



[심층 분석 - 방 번호 : 1177]

동시성 제어가 없는 '읽기-수정-쓰기' 과정에서 데이터가 어떻게 오염되는지를 증명하기 위해, 1177번 방에서 발생한 '값 불일치(Lost Update)' 오류 로그를 집중 분석하였다. 분석 결과, 해당 방에서만 총 19건의 값 불일치 오류를 확인하였다. 오류로 인해 발생한 데이터의 오차 값은 대부분 '1' 수준이었으나, 후반부로 갈수록 경합이 심화되며 오차가 '2'까지 벌어지는 심각한 오류도 2건 발견되었다. 또한, 오류 발생 빈도는 시간의 흐름에 따른 테스트 구간별로 점진적인 증가 추이를 보였다. 초기 구간에서는 1건에 불과했으나, 마지막 구간에서는 8건으로 급증하여 시스템 부하와 오류 발생 간의 상관관계를 시사한다.

이러한 값 불일치 현상은 '읽기-수정-쓰기'의 전체 과정이 원자적(Atomic)으로 보장되지 않았기 때문에 발생한 직접적인 결과이다. 결론적으로, 이 분석은 동시성 제어의 부재가 데이터의 정합성을 어떻게 파괴하는지 실증적으로 보여준다. 따라서 안정적인 백엔드 시스템은 임계 구역 내의 모든 연산이 원자적으로 처리되도록 보장하는 동기화 메커니즘이 필수적이라는 결론을 가진다.

동시성 시나리오 테스트 및 결과 분석

'Check-Then-Act' 구조의 동시성 결함 실증 - 규칙 2: 경합 발생(Contention) 분석

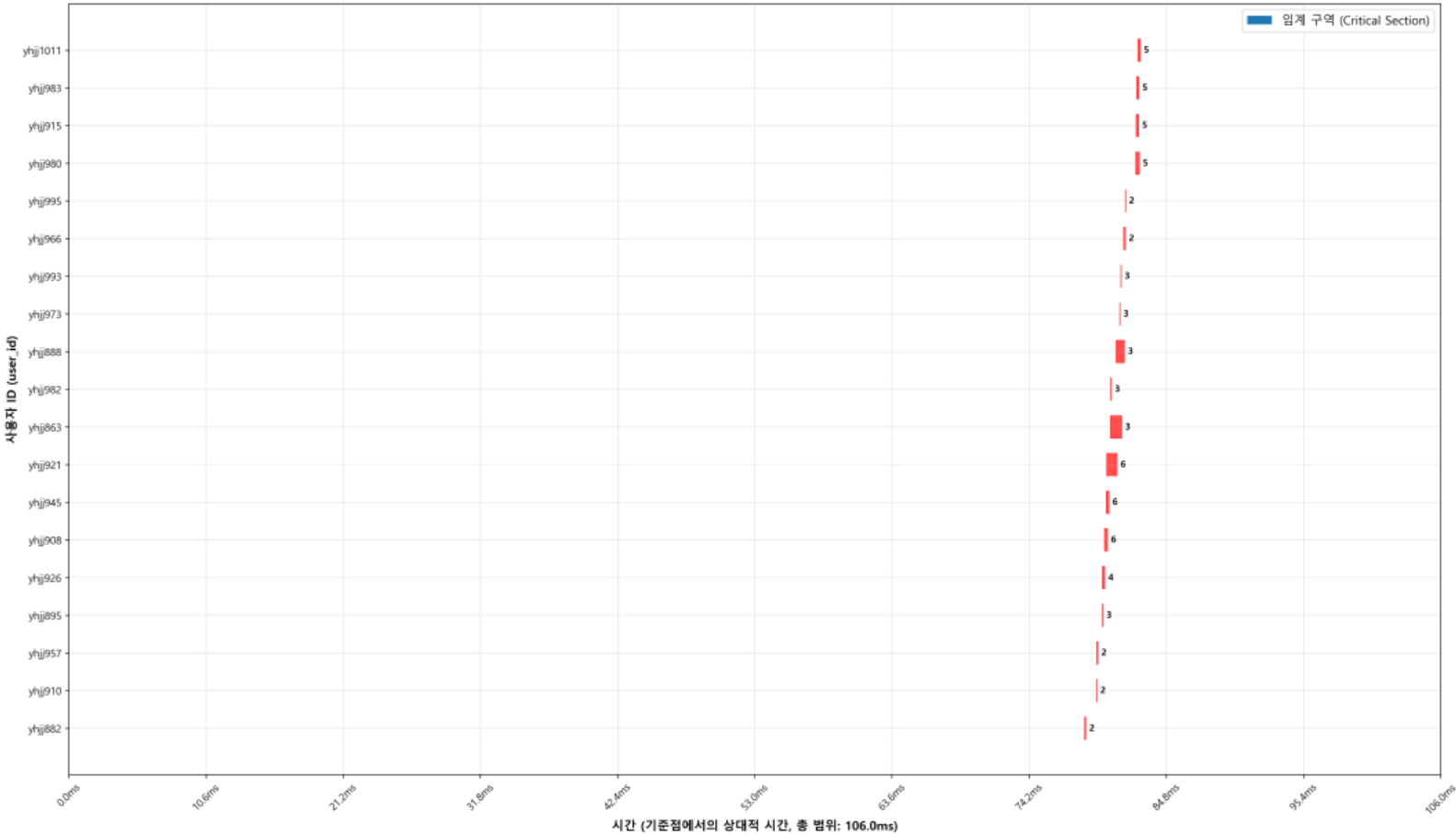
본 절은 'Check-Then-Act' 구조에서 임계 구역(Critical Section) 보호가 실패하여 여러 스레드가 동시에 같은 자원에 접근하는 '경합 발생(Contention)' 현상을 간트 차트(Gantt Chart)로 시각화하여 증명한다
차트의 각 수평 막대는 개별 스레드가 임계 구역을 점유한 시간을 나타내며, 이 막대들이 시간 축 위에서 겹치는 모든 구간이 바로 동시성 제어가 실패한 Race Condition의 발생 지점이며 이를 통해 단순히 오류 발생 여부를 넘어, 어떤 스레드들이, 언제, 얼마나 오랫동안 충돌했는지를 직관적으로 분석하고 동시성 문제의 심각도를 정량적으로 파악할 수 있습니다.

[표 – 계측 데이터]

방 번호	구간	사용자 ID	경합 발생 스레드 개수	경합 발생 사용자 ID 들	임계구역 시작 지점 시각	임계구역 종료 시점 시각	임계구역 유지 시간	선별 사유
1179	1	yhjj828	2	yhjj839, yhjj828	0.000ms	0.152ms	152.0ns	구간 내 임계구역 최대 지속 시간
1179	1	yhjj839	2	yhjj839, yhjj828	0.019ms	0.061ms	42.0ns	구간 내 임계구역 최소 지속 시간
1179	2	yhjj925	2	yhjj856, yhjj925	41.535ms	41.640ms	104.4ns	구간 내 임계구역 최대 지속 시간
1179	2	yhjj856	2	yhjj856, yhjj925	41.549ms	41.622ms	73.4ns	구간 내 임계구역 최소 지속 시간
1179	3	yhjj970	2	yhjj907, yhjj970	64.815ms	65.072ms	256.5ns	구간 내 임계구역 최대 지속 시간
1179	3	yhjj907	2	yhjj907, yhjj970	64.940ms	64.985ms	44.5ns	구간 내 임계구역 최소 지속 시간
1179	4	yhjj1017	3	yhjj1012, yhjj1017, yhjj845	76.837ms	76.986ms	149.1ns	구간 내 임계구역 최대 지속 시간
1179	4	yhjj845	3	yhjj1012, yhjj1017, yhjj845	76.850ms	76.917ms	67.1ns	구간 내 임계구역 최소 지속 시간
1179	5	yhjj945	6	yhjj921, yhjj908, yhjj945, yhjj863, yhjj982, yhjj888	80.145ms	80.381ms	236.2ns	구간 내 임계구역 최대 지속 시간
1179	5	yhjj921	6	yhjj888, yhjj921, yhjj863, yhjj973, yhjj993, yhjj966	80.189ms	81.005ms	816.1ns	구간 내 임계구역 최소 지속 시간

[데이터 분석]

규칙 2: 경합 발생 간트 차트 - 방 1179, bin 5 (실제 시간 위치)



[심층 분석 : 구간 5]

동시성 제어가 없는 'Check-Then-Act' 구조에서 자원 경합이 필연적으로 발생하는지를 증명하기 위해 방 번호 1179의 로그 데이터를 심층 분석하였다. 분석 결과, 총 116건의 임계 구역충돌이 발견되었으며, 평균 2.94개의 스레드가 동일 자원을 놓고 경쟁하였다. 특히 가장 심각한 경우에는 최대 7개의 스레드가 동시에 임계 구역에 진입하는 등, 시스템이 잠재적 데이터 오염 위험에 직접적으로 노출되었음을 정량적으로 확인하였다.

이는 '자원 확인(Check)'과 '상태 변경(Act)' 로직이 원자적으로 실행되지 않아 발생한 직접적인 결과이다. 결론적으로, 경합의 발생은 그 자체로 'Lost Update'나 '정원 초과'와 같은 심각한 데이터 정합성 문제의 전조 증상을 의미하며 이 분석을 통해 안정적인 백엔드 시스템을 설계하기 위해서는 모든 임계 구역을 식별하고, Lock이나 Semaphore 등의 동기화 메커니즘을 통해 자원 접근을 반드시 제어해야 한다는 결론을 가진다.

동시성 시나리오 테스트 및 결과 분석

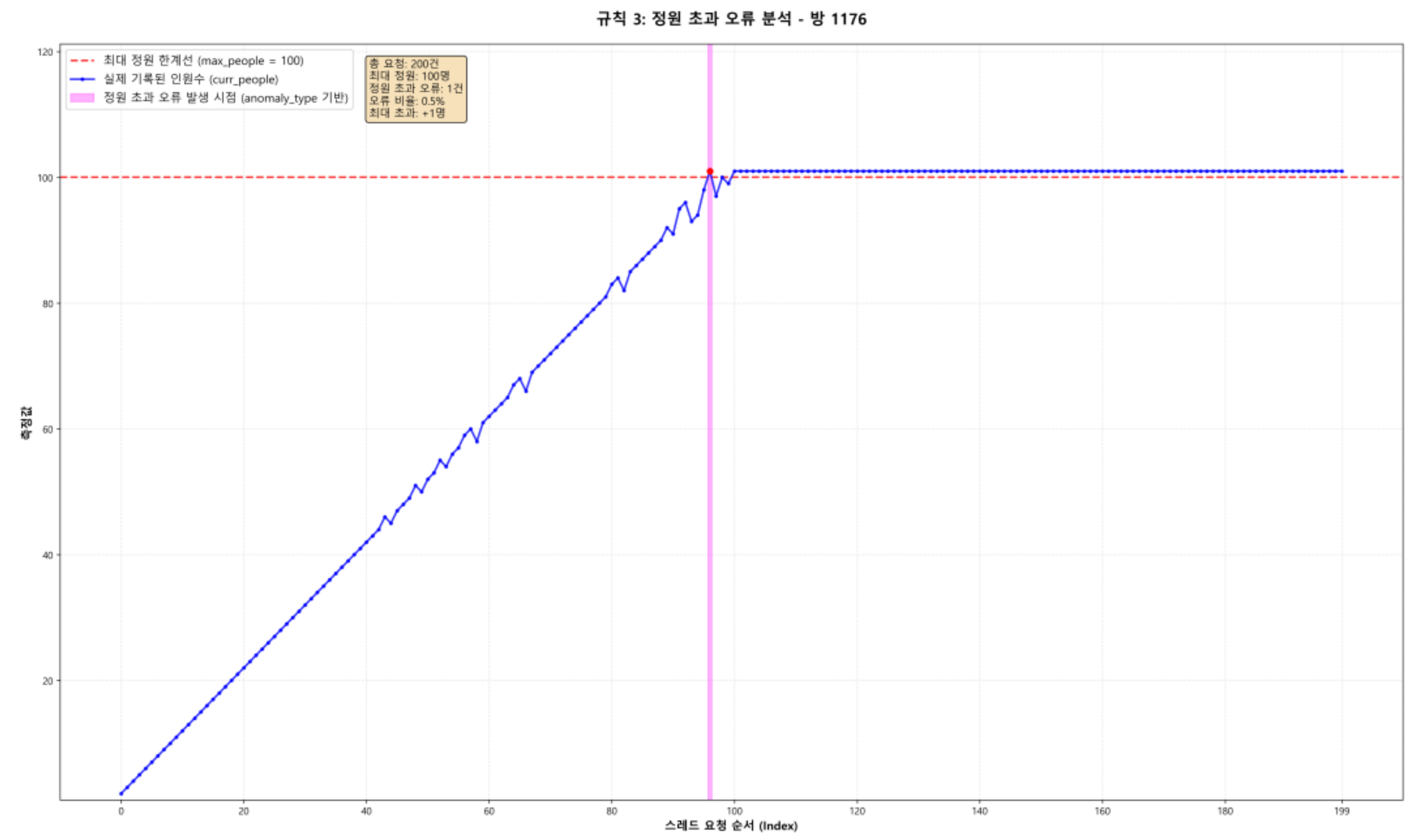
'Check-Then-Act' 구조의 동시성 결함 실증 - 규칙 3 : 정원 초과 오류

본 절은 'Check-Then-Act' 구조의 동시성 결함이 어떻게 시스템의 비즈니스 규칙(Business Rule)을 파괴하는지를 '정원 초과' 현상을 통해 실증적으로 분석한다. 시각화 차트는 시간의 흐름에 따른 '실제 기록 인원수'의 변화와, 시스템이 허용하는 '최대 정원 한계선'을 함께 표시하여 두 데이터의 관계를 비교한다. 차트의 '최대 정원 한계선'을 '실제 기록 인원수'가 초과하는 모든 지점이 바로 동시성 제어 실패로 인해 비즈니스 규칙이 위반된 명백한 증거이다. 이를 통해 단순히 오류 발생 여부를 넘어, 어느 시점에서, 얼마나 많은 인원이 정원을 초과했는지 직관적으로 추적하고 시스템의 안정성 훼손 수준을 정량적으로 파악할 수 있다.

[표 – 계측 데이터]

구간	전체 요청 수	발생 건수	발생 률(%)	총 초과 인원	평균 초과 인원	최소 초과 인원	최대 초과 인원	중간값 초과 인원
1	20	0	0	0	0	0	0	0
2	20	0	0	0	0	0	0	0
3	20	0	0	0	0	0	0	0
4	20	0	0	0	0	0	0	0
5	20	1	5	1	1	1	1	1
6	20	0	0	0	0	0	0	0
7	20	0	0	0	0	0	0	0
8	20	0	0	0	0	0	0	0
9	20	0	0	0	0	0	0	0
10	20	0	0	0	0	0	0	0

[데이터 분석]



[심층 분석 : 구간 5]

동시성 제어가 없는 환경에서 '조회 후 처리(Check-Then-Act)' 로직이 어떻게 데이터 정합성을 파괴하는지 실증적으로 분석하였다.

가설은 다음과 같다.

"여러 스레드가 동시에 '방에 여유가 있는지 확인(Check)'하고 '입장(Act)'한다면, 정원(Max People)을 초과하는 심각한 비즈니스 규칙 위반이 발생할 것이다."

[분석 과정 및 결과]

각기 다른 채팅방(ID: 1~10)에 20개의 동시 입장 요청을 보내는 테스트를 설계하고, 그 결과를 집계했다.

분석 결과, 대부분의 방에서는 오류가 발생하지 않았으나 5번 방에서 5%의 확률(20개 요청 중 1건)로 정원 초과 오류가 발생함을 확인했습니다. 이 단 한 번의 오류가 왜, 어떻게 발생했는지 심층적으로 분석함.

동시성 시나리오 테스트 및 결과 분석

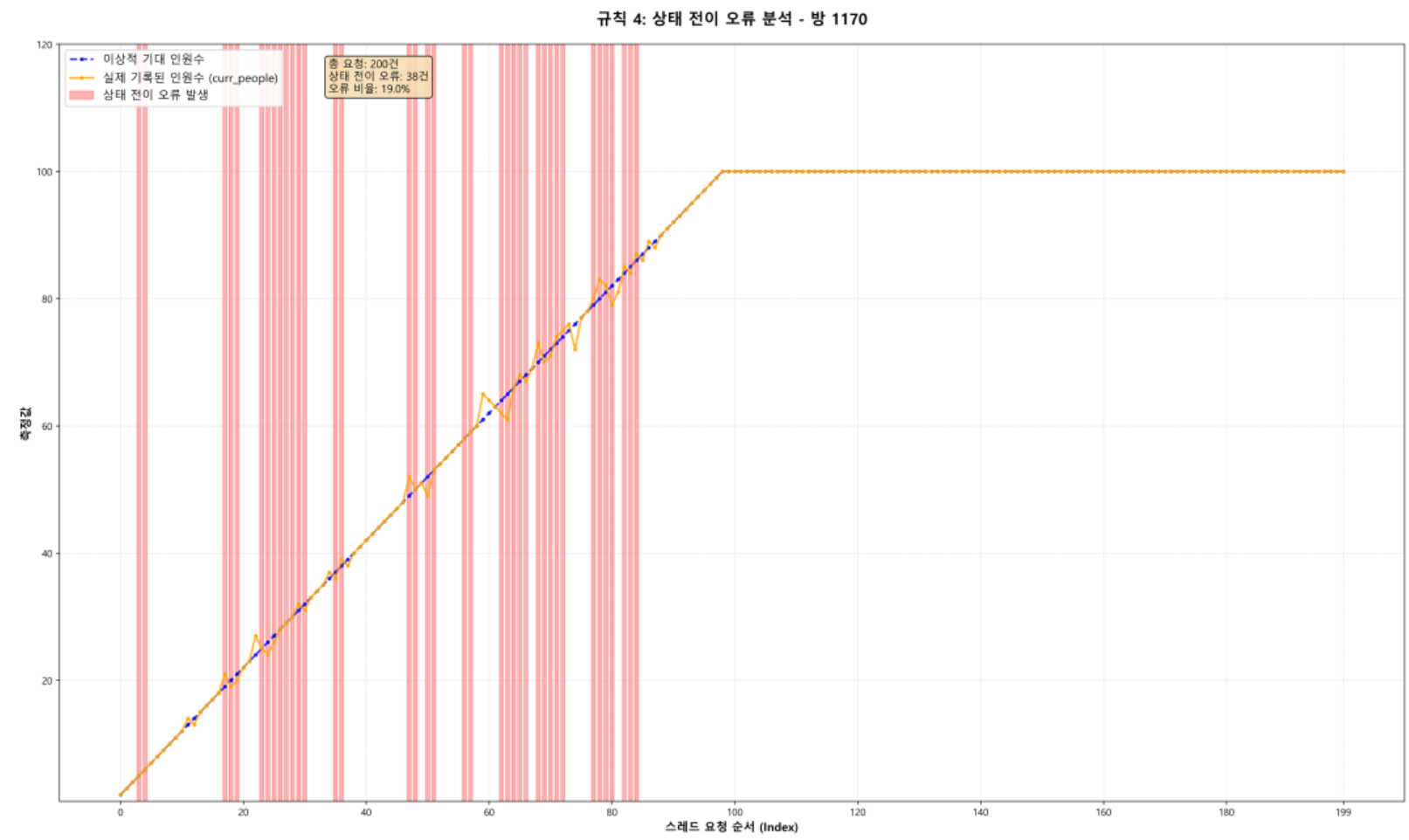
'Check-Then-Act' 구조의 동시성 결함 실증 - 규칙 4: 상대 전이 오류

본 절은 경쟁 상태가 없는 이상적인 환경에서의 데이터 증가 패턴과, 실제 부하 상황에서 기록된 데이터의 변화를 비교하여 '상태 전이 오류'를 분석한다. 시각화 차트는 시간 순서에 따라 순차적으로 데이터가 증가하는 '이상적 기대치'의 흐름과, 실제 동시성 환경에서 최종 기록된 '현실의 상태값' 변화를 하나의 그래프에 중첩하여 표현한다. 두 흐름이 서로 일치하지 않고 벌어지는 모든 지점은 '상태 전이 오류'의 증거이다. 이는 특정 스레드가 자신의 처리 순서에 맞는 최신 상태를 읽지 못하고 연산을 수행한 경우뿐만 아니라, 정상적인 최신 값을 읽었더라도 그 연산 결과를 최종 기록하기 전에 다른 스레드가 먼저 상태를 변경하여 데이터의 순차적 일관성을 훼손시킨 경우에도 발생한다. 즉, 이 분석은 '읽기-수정-쓰기'의 전체 과정이 원자적으로 보장되지 않을 때, 시스템의 상태 변화 과정 자체에 결함이 발생함을 정량적으로 증명한다.

[표 – 계측 데이터]

방 번호	구간	스레드 요청 순서	기대 인원 수	실제 인원 수	오차 수
1170	1	4	5	6	1
1170	1	18	19	21	2
1170	1	19	20	19	-1
1170	1	20	21	20	-1
1170	2	25	26	25	-1
1170	2	26	27	30	3
1170	2	27	28	32	4
1170	2	28	29	27	-2
1170	2	29	30	28	-2
1170	5	85	86	84	-2

[데이터 분석]



[심층 분석 : 방 1170 번]

본 절은 경쟁 상태가 없는 이상적 환경에서의 데이터 증가 패턴과, 실제 부하 상황에서 기록된 데이터의 변화를 비교하여 '상태 전이 오류'를 분석한다. 시각화 차트는 시간 순서에 따라 순차적으로 데이터가 증가하는 '이상적 기대치'의 흐름과, 실제 동시성 환경에서 최종 기록된 '현실의 상태값' 변화를 하나의 그래프에 중첩하여 표현한다.

두 흐름이 서로 일치하지 않고 벌어지는 모든 지점은 '상태 전이 오류'의 증거이다. 이는 개별 스레드의 '읽기-수정-쓰기' 연산이 전체적으로 원자성을 보장받지 못하기 때문에 발생하는 직접적인 결과이다. 즉, 특정 스레드가 연산을 시작하는 시점에 읽은 데이터가 이미 오래된 값이거나(Stale Read), 혹은 데이터를 읽고 쓰는 사이에 다른 스레드가 개입하여 상태를 먼저 변경함으로써(Race Condition), 최종적으로 기록되는 값이 시간적 순서에 맞지 않게 되는 것이다.

동시성 시나리오 테스트 및 결과 분석

'Check-Then-Act' 동시성 제어 기법에 따른 성능 트레이드오프 분석

백엔드 시스템의 안정성을 저해하는 경쟁 상태 문제를 데이터에 기반하여 증명하고, 이를 해결하기 위한 각 동시성 제어 기법들의 성능 특성과 트레이드오프 관계를 정량적으로 분석하였다. 동기화 제어가 없는 IF-ELSE 구조의 성능을 10개의 테스트 방에 걸쳐 측정한 결과, 단일한 실패 패턴이 아닌, 각기 다른 유형의 성능 병목 현상이 예측 불가능하게 발생하는 것을 확인하였다., 이후 진행될 모든 동기화 기법의 성능 오버헤드와 효율성을 비교하기 위한 객관적인 기준선을 수립한다.

[동시성 결함 탐지 결과 요약 - 방 별 집계]

방 번호	전체 요청 수	요청 성공률	용량 초과 실패율	요청 성공 평균 대기 시간 (μs)	요청 성공 평균 작업 시간 (μs)	용량 초과 실패 평균 대기시간 (μs)	용량 초과 평균 작업 시간 (μs)
1170	200	49.50%	50.50%	128,592	456,527	39,807	87,476
1171	200	49.50%	50.50%	44,575	108,349	41,272	106,307
1172	200	49.50%	50.50%	48,146	90,779	82,908	144,667
1173	200	49.50%	50.50%	42,421	101,077	184,333	331,229
1174	200	49.50%	50.50%	29,530	96,037	67,830	199,023
1175	200	49.50%	50.50%	34,685	99,532	84,334	103,727
1176	200	50.00%	50.00%	58,554	149,122	72,020	86,233
1177	200	49.50%	50.50%	30,469	91,804	117,658	94,314
1178	200	49.50%	50.50%	38,344	109,353	37,271	105,331
1179	200	49.50%	50.50%	28,267	103,313	103,974	197,131

위 표는 10개의 테스트 방에서 측정한 핵심 성능 지표를 요약한 것이다. 주목할 점은, '가장 성능이 나쁜 방'을 하나의 지표로 특정할 수 없다는 점이다. 특정 방은 임계 구역 진입 전 평균 대기 시간이 가장 길었던 반면, 다른 방은 임계 구역 내 평균 체류 시간이 가장 길게 나타나는 등, 각 방이 서로 다른 유형의 성능 병목을 무작위적으로 겪고 있음을 보여준다. 이는 동기화 제어가 없는 시스템의 성능이 얼마나 예측 불가능한지를 증명하는 결과이다.

[문제 구간 특정 - 방 1170]

방 번호	구간	전체 요청 수	요청 성공률	용량 초과 실패율	요청 성공 평균 대기 시간 (μs)	요청 성공 평균 작업 시간 (μs)	용량 초과 실패 평균 대기시간 (μs)	용량 초과 평균 작업 시간 (μs)
1170	1	20	100 %	0 %	150.76	505.34	0	0
1170	5	20	95 %	5 %	74.8368	265.66	17	19.2
1170	6	20	0	100%	0	0	30.255	35.28
1170	10	20	0	100%	0	0	46.255	128.28

Per_Room_Stats에서 확인된 일관된 50%의 실패율이 시간의 흐름에 따라 어떻게 형성되는지 추적하기 위해, 대표적인 방들의 구간 별 데이터를 심층 분석하였다. 분석 결과, 모든 방에서 매우 유사한 패턴이 발견되었다. 테스트 초기 구간(1~4)에서는 요청 성공률이 100%에 육박하며 안정적인 모습을 보였으나, 정확히 bin 5 시점에서 95%로-소폭 하락하며 첫 번째 실패가 발생하였다. 그리고 그 직후인 bin 6부터는 성공률이 0%로 급락하며 시스템이 완전한 '비생산적 점유' 상태로 고착되는 현상을 보였다. 이는 IF-ELSE 구조가 특정 부하 임계점을 넘어서는 순간, 급격한 성능 붕괴를 겪게 됨을 명확히 보여준다.으로 겪고 있음을 보여준다. 이는 동기화 제어가 없는 시스템의 성능이 얼마나 예측 불가능한지를 증명하는 결과이다.

[분석 결과]

제어 없는 IF-ELSE 구조의 성능 측정 결과, 시스템의 근본적인 비효율과 예측 불가능성을 증명하는 두 가지 주요 현상을 확인하였다.

- 첫째, 전체 요청의 약 50%가 '용량 초과'로 실패 처리되는 높은 자원 낭비율을 보였다. 이는 임계 구역에 진입했음에도 아무 작업을 수행하지 못하고 나온 '비생산적 점유'가 빈번함을 의미하며, 시스템 리소스의 절반이 잠재적으로 낭비되고 있음을 의미한다.
- 둘째, 성공한 요청과 실패한 요청 모두에서 성능 편차가 매우 크게 나타났다. 각 방의 구간 별 데이터를 심층 분석한 결과, '대기 시간'과 '임계 구역 체류 시간' 모두 명확한 규칙성 없이 불규칙하게 측정되었다. 이는 명시적인 락 경합이 없음에도 OS 스케줄링 등 외부 요인에 의해 시스템 성능이 무작위적으로 좌우되는, 제어 불가능한 상태임을 증명한다.

결론적으로 IF-ELSE 구조는 높은 자원 낭비율과 극심한 성능 변동성을 동시에 보여주는, 가장 비효율적이고 불안정한 방식이다. 이 측정 결과는 이후 분석할 동기화 기법들이 이러한 '낭비'와 '불확실성'을 얼마나 효과적으로 제어하는지 평가하는 핵심 기준선이 된다.

동시성 시나리오 테스트 및 결과 분석

'Synchronized' 구조의 동시성 결함 테스트 - RaceCondtion 규칙 1 : 값 불일치

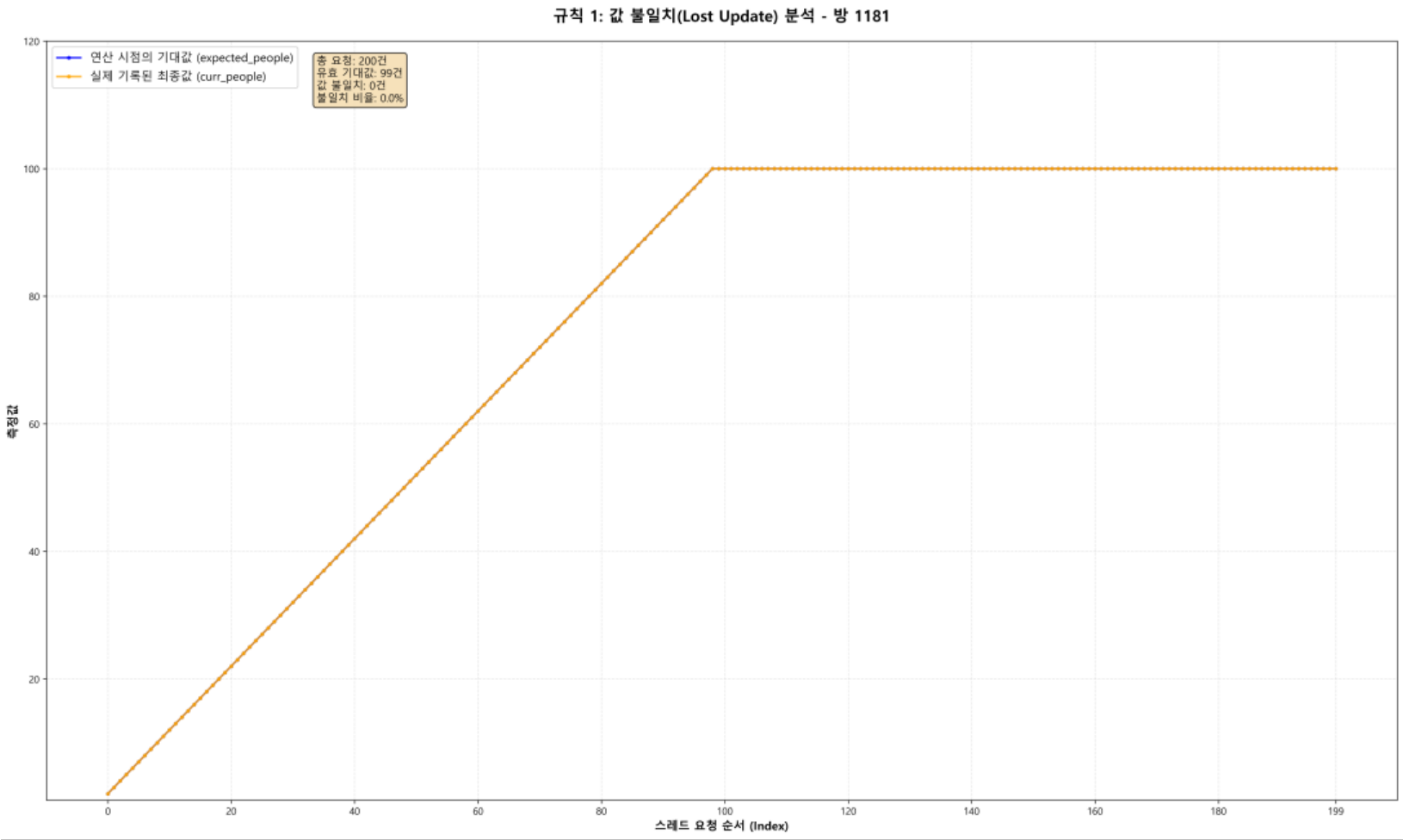
IF-ELSE 구조의 분석을 통해, 명시적인 동기화 제어가 없는 시스템의 데이터 정합성 훼손을 증명하였다. 본 절은 이에 대한 가장 기본적인 해결책인 Java의 synchronized 키워드를 적용하여, 동일한 부하 조건에서 정합성 문제가 해결되는지 검증하는 것을 목표로 한다. 더 나아가, 정합성을 확보하는 대가로 발생하는 성능 오버헤드(Overhead)를 정량적으로 측정하고, 그 성능적 특성을 분석한다. 이를 위해 임계 구역(Critical Section) 전체를 synchronized 블록으로 감싼 후, 이전과 동일한 조건으로 10회의 부하 테스트를 반복 수행하였다. 아래 데이터는 synchronized가 어떻게 '값 불일치'와 같은 오류를 원천적으로 방지하는지, 그리고 이 과정에서 잠금(Lock) 메커니즘이 유발하는 성능 변화를 구체적으로 보여준다.

[표 – 계측 데이터]

방 번호	bin	순번	사용자 ID	이전 인원	기대인원	현재인원	오차
1181	1	1	yhjj831	1	2	2	0
1181	1	2	yhjj824	2	3	3	0
1181	1	3	yhjj856	3	4	4	0
1181	1	4	yhjj819	4	5	5	0
1181	1	5	yhjj820	5	6	6	0
1181	1	6	yhjj887	6	7	7	0
1181	1	7	yhjj875	7	8	8	0
1181	1	8	yhjj878	8	9	9	0
1181	1	9	yhjj879	9	10	10	0
1181	1	10	yhjj829	10	11	11	0

[데이터 분석]

[차트]



[심층 분석 - 방 번호 : 1181]

이전 IF-ELSE 구조 테스트에서는 prev_people(이전 인원)이 84일 때 curr_people(현재 인원)이 87로 기록되는 등 총 19건의 '값 불일치'가 발생했었다. 본 절은 synchronized 키워드를 통해 이 문제가 해결되는지를 검증하였다.

동일한 부하 조건에서 테스트를 진행한 결과, synchronized 블록에 의해 임계 구역이 보호됨에 따라 모든 스레드가 해당 구간을 순차적으로 진입하고 실행하는 것을 데이터로 확인하였다. 제공된 데이터 샘플을 보면, prev_people이 1일 때 curr_people은 정확히 2가 되고, 다음 스레드는 그 결과값인 2를 prev_people로 읽어 curr_people을 3으로 만드는 등, 모든 연산이 이전 스레드의 작업 결과를 정확히 이어받아 순차적으로 처리되었다.

그 결과, 단 한 건의 '값 불일치' 오류도 발생하지 않았으며(오류 발생률 0%), 이를 통해 synchronized 키워드가 '읽기-수정-쓰기' 연산의 원자성(Atomicity)을 보장하여 데이터 정합성을 완벽하게 확보함을 증명하였다.

동시성 시나리오 테스트 및 결과 분석

'Synchronized' 구조의 동시성 결함 테스트 - RaceCondtion 규칙 2: 경합 발생(Contention) 분석

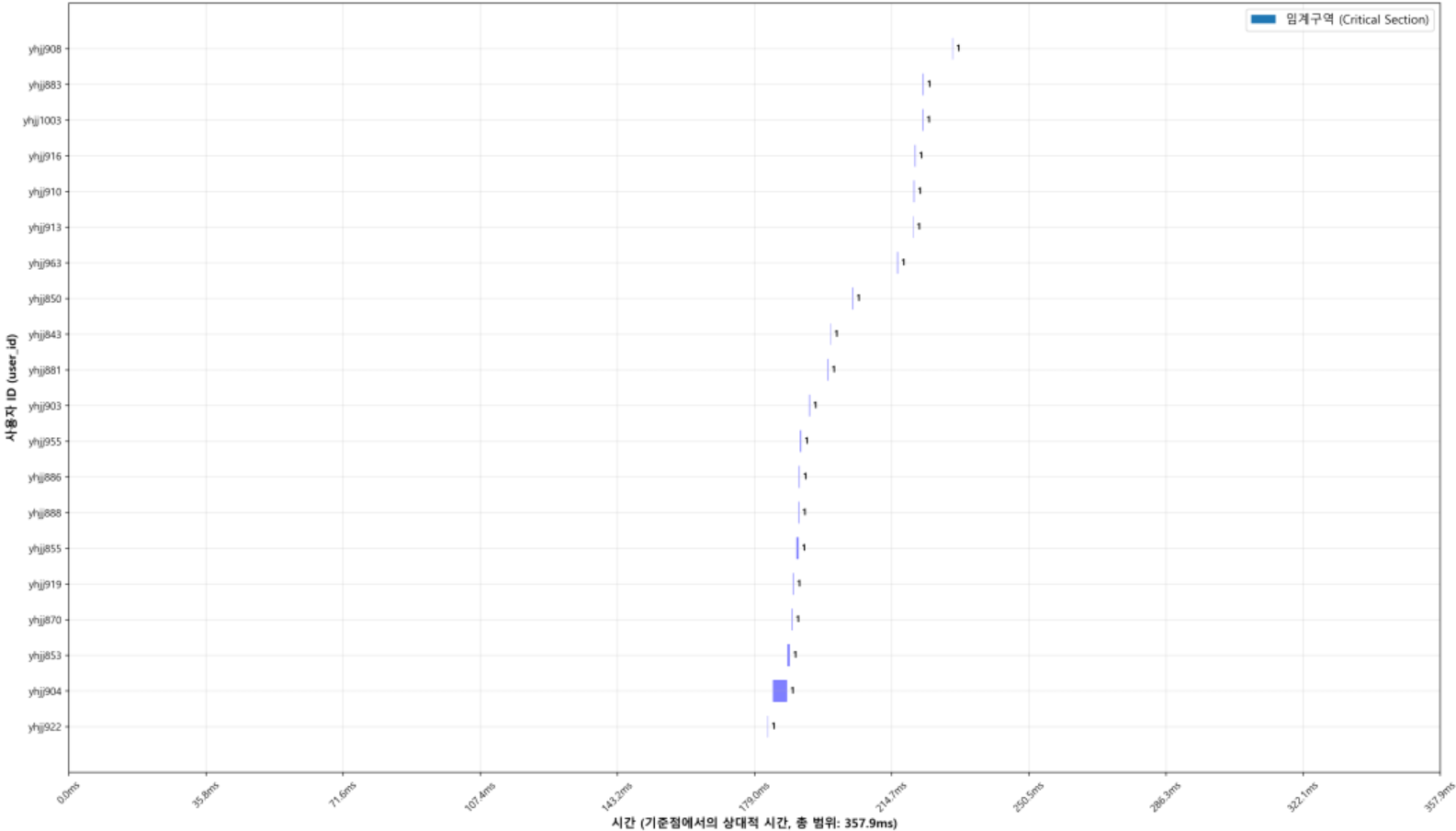
앞선 '값 불일치' 분석을 통해 synchronized 키워드가 데이터 정합성을 완벽하게 보장함을 증명하였다. 그러나 이 정합성은 '상호 배제(Mutual Exclusion)'라는 메커니즘을 통해 달성되며, 이는 필연적으로 성능 비용(Overhead)을 수반한다. 본 절은 바로 이 비용의 실체를 '경합(Contention)'이라는 관점에서 심층적으로 분석하는 것을 목표로 한다. 여기서 경합은 IF-ELSE 구조에서와 같은 '동시 실행'이 아니라, 하나의 잠금(Lock)을 획득하기 위해 여러 스레드가 순서를 기다리며 발생하는 '병목 현상'을 의미한다. 이 테스트는 각 스레드가 임계 구역에 진입하기 위해 얼마나 오래 대기했는지, 그리고 임계 구역 내부에서 얼마나 머물렀는지를 나노초(ns) 단위로 측정하여, synchronized 잠금이 유발하는 성능 저하의 규모와 패턴을 정량적으로 분석한다.

[표 – 계측 데이터]

방 번호	구간	사용자 ID	경합 발생 스레드 개수	경합 발생 사용자 ID 들	임계구역 시작 지점 시각	임계구역 종료 시점 시각	임계구역 유지 시간
1181	5	yhjj922	1	yhjj922	0.000ms	0.045ms	45500ns
1181	5	yhjj904	1	yhjj904	1.467ms	4.993ms	3525700ns
1181	5	yhjj853	1	yhjj853	5.250ms	5.639ms	388500ns
1181	5	yhjj870	1	yhjj870	6.282ms	6.360ms	77700ns
1181	5	yhjj919	1	yhjj919	6.622ms	6.707ms	85100ns
1181	5	yhjj855	1	yhjj855	7.617ms	7.982ms	365000ns
1181	5	yhjj888	1	yhjj888	8.019ms	8.086ms	66500ns
1181	5	yhjj886	1	yhjj886	8.211ms	8.282ms	71200ns
1181	5	yhjj955	1	yhjj955	8.557ms	8.661ms	104200ns
1181	5	yhjj903	1	yhjj903	10.864ms	10.925ms	61000ns

[데이터 분석]

규칙 2: 전체 임계구역 간트 차트 - 방 1181, bin 5 (모든 구간)



[심층 분석 : 구간 5]

데이터 분석 결과, contention_group_size(경합 그룹 크기)는 모든 요청에 대해 항상 '1'로 측정되었다. 이는 synchronized에 의해 임계 구역 내에는 어떤 순간에도 단 하나의 스레드만 존재하도록 보장되었음을 의미한다. 즉, 데이터 정합성을 훼손할 수 있는 동시 실행 경합이 원천적으로 차단되었음을 정량적으로 증명하는 결과이다.

동시성 시나리오 테스트 및 결과 분석

'Synchronized' 구조의 동시성 결함 테스트 - RaceCondtion 규칙 3 : 정원 초과 오류

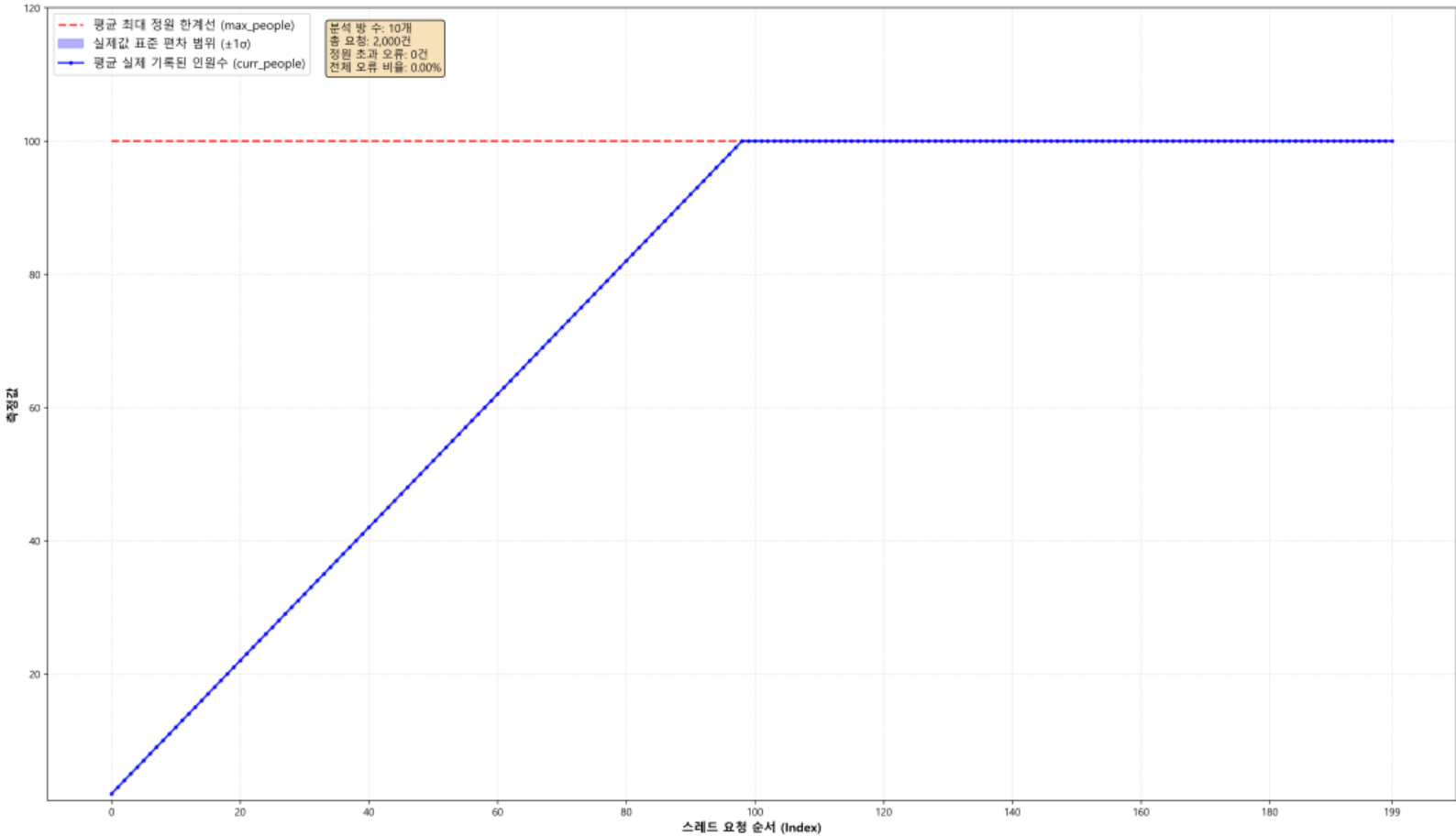
앞선 IF-ELSE 구조 테스트에서는, 시스템 부하가 임계점에 도달했을 때 모든 요청이 '용량 초과'로 실패하며 요청 성공률이 0%로 수렴하는 장애 수준의 문제가 발생하였다. 본 절은 synchronized 키워드가 이러한 심각한 비즈니스 규칙 위반 문제를 해결할 수 있는지 데이터로 검증하는 것을 목표로 한다.

[표 - 계측 데이터]

방 번호	전체 요청 수	발생 건수	발생 률(%)	총 초과 인원	평균 초과 인원	최소 초과 인원	최대 초과 인원	중간값 초과 인원
1181	200	0	0	0	0	0	0	0
1182	200	0	0	0	0	0	0	0
1183	200	0	0	0	0	0	0	0
1184	200	0	0	0	0	0	0	0
1185	200	0	0	0	0	0	0	0
1186	200	0	0	0	0	0	0	0
1187	200	0	0	0	0	0	0	0
1188	200	0	0	0	0	0	0	0
1189	200	0	0	0	0	0	0	0
1190	200	0	0	0	0	0	0	0

[데이터 분석]

규칙 3: 정원 초과 오류 분석 - 전체 10개 방 평균 및 실제 값 표준편차 범위



[심층 분석 : 모든 방]

이전 IF-ELSE 구조 테스트에서는 시스템 부하가 임계점에 도달했을 때, 모든 요청이 '용량 초과'로 실패하며 요청 성공률이 0%로 수렴하는 장애 상태가 관측되었다. 본 테스트는 synchronized 키워드가 이 문제를 해결하는지 검증하는 것을 목표로 한다.

데이터 테이블의 결과와 같이, synchronized를 적용한 모든 테스트 방에서는 단 한 건의 '정원 초과' 오류도 발생하지 않았으며, 요청 성공률 100%를 안정적으로 유지하였다.

이는 synchronized가 상호 배제(Mutual Exclusion) 원칙에 따라, 한 번에 단 하나의 스레드만 임계 구역에 진입하도록 허용하기 때문이다. 이러한 독점적 실행이 보장되므로, '인원수 확인(Check)'과 '인원수 증가(Act)' 사이를 다른 스레드가 비집고 들어올 수 없게 된다. 결과적으로, 이 두 단계의 로직은 외부에서 볼 때 마치 분리할 수 없는 하나의 연산처럼 동작하는 원자성(Atomicity)을 확보하게 되어 정합성 문제를 해결하는 것이다.

동시성 시나리오 테스트 및 결과 분석

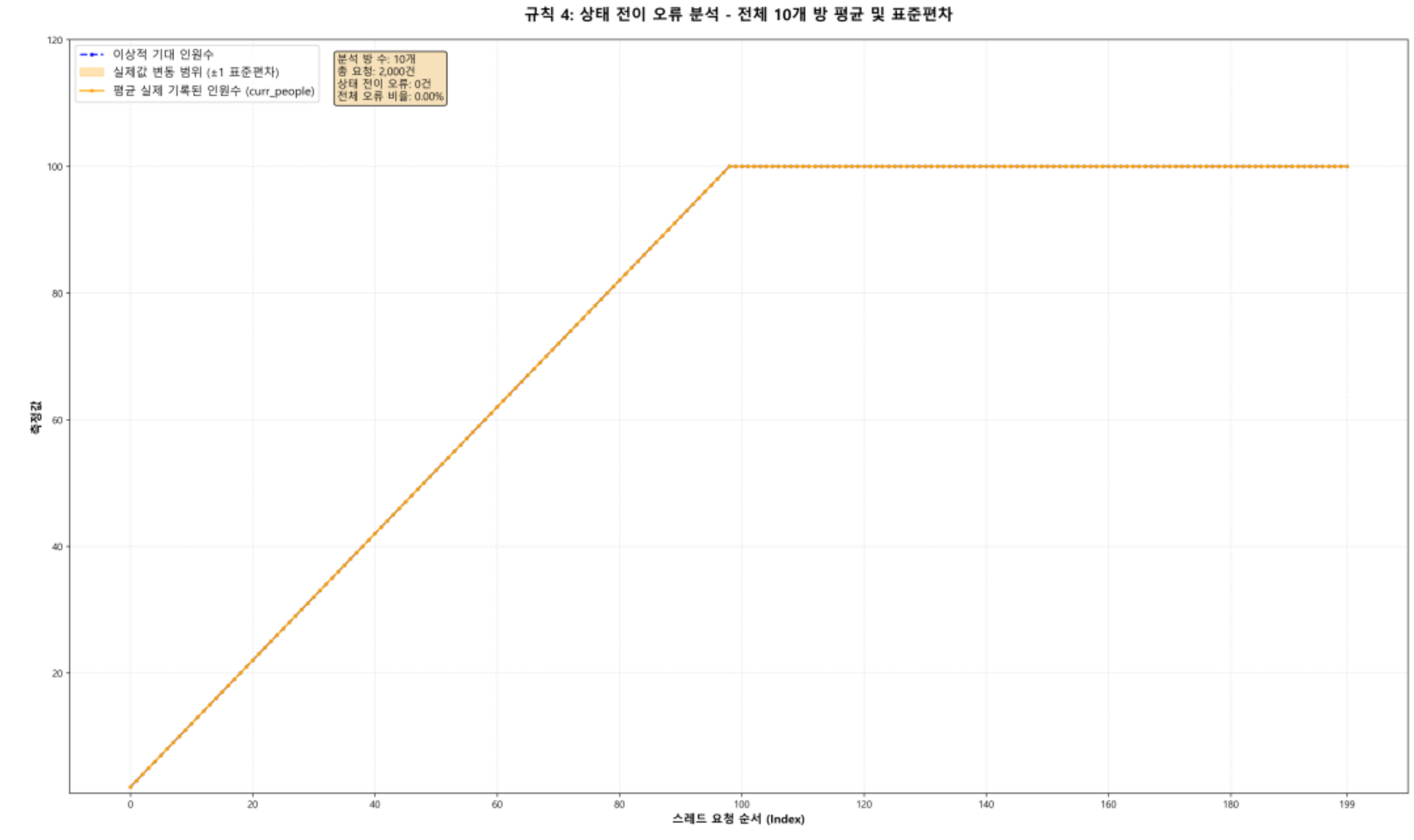
'Synchronized' 구조의 동시성 결함 테스트 - RaceCondtion 규칙 4: 상대 전이 오류

앞선 IF-ELSE 구조에서는 실제 기록값이 이상적인 기대치를 따라가지 못하고 불규칙하게 변동하는 '상태 전이 오류'가 다수 발생하였다. 본 절은 synchronized 키워드가 스레드 실행 순서를 제어하여, 이러한 데이터의 순차적 일관성 문제를 해결할 수 있는지 검증하는 것을 목표로 한다.

[표 – 계측 데이터]

방 번호	전체 요청 수	발생 건수	발생율(%)	총합 값	평균 값	최소 값	최대 값	중간 값
1181	200	0	0	0	0	0	0	0
1182	200	0	0	0	0	0	0	0
1183	200	0	0	0	0	0	0	0
1184	200	0	0	0	0	0	0	0
1185	200	0	0	0	0	0	0	0
1186	200	0	0	0	0	0	0	0
1187	200	0	0	0	0	0	0	0
1188	200	0	0	0	0	0	0	0
1189	200	0	0	0	0	0	0	0
1190	200	0	0	0	0	0	0	0

[데이터 분석]



[심층 분석 : 모든 방]

앞선 IF-ELSE 구조에서는 실제 기록값이 이상적인 기대치를 따라가지 못하고 불규칙하게 변동하는 '상태 전이 오류'가 다수 발생하였다. 본 절은 synchronized 키워드가 스레드 실행 순서를 제어하여, 이러한 데이터의 순차적 일관성 문제를 해결할 수 있는지 검증하는 것을 목표로 한다.

제시된 '방별 규칙 4 통합 분석' 데이터에서 볼 수 있듯, synchronized를 적용한 모든 테스트 방에서는 단 한 건의 '상태 전이 오류'도 발생하지 않았으며, 오류 발생률은 0%로 측정되었다.

이는 synchronized가 모든 스레드가 임계 구역을 순서대로 진입하도록 강제함으로써, 각 요청이 이전 요청의 최종 상태를 정확히 기반하여 다음 상태를 갱신하도록 보장하기 때문이다. 이를 통해, 모든 요청의 '실제 기록 인원수'는 '이상적 기대치'와 완벽하게 일치하였으며, 이는 synchronized가 데이터의 최종 값뿐만 아니라, 그 값이 만들어지는 과정의 순차성까지 보장하는 강력한 메커니즘임을 증명한다.

동시성 시나리오 테스트 및 결과 분석

'Synchronized' 동시성 제어 기법에 따른 성능 트레이드오프 분석 - 비 이중 확인 구조

앞선 IF-ELSE 구조가 데이터 정합성을 보장하지 못하고, 예측 불가능한 성능을 보이는 문제를 확인하였다. 본 절은 Java의 가장 기본적인 동기화 기법인 synchronized를 적용하여 이 문제를 해결하고, 그 과정에서 발생하는 성능 비용(Overhead)을 정량적으로 측정하는 것을 목표로 한다. synchronized는 '읽기-수정-쓰기' 연산의 원자성(Atomicity)을 보장하여 데이터의 무결성을 확보하는 강력한 수단이다. 하지만 잠금(Lock)을 획득하고 해제하는 과정은 필연적으로 시스템에 부하를 유발한다. 따라서 본 분석에서는 synchronized가 정합성을 100% 보장하는 것을 데이터로 증명하는 동시에, 기준선(IF-ELSE) 대비 어느 정도의 성능 오버헤드가 발생하는지를 핵심 지표를 통해 심층적으로 분석한다.

[동시성 결함 탐지 결과 요약 - 방 별 집계]

방 번호	전체 요청 수	요청 성공률	용량 초과 실패율	요청 성공 대기 시간 평균 (μs)	요청 성공 대기 시간 총합 (μs)	요청 성공 작업 시간 평균 (μs)	요청 성공 작업 시간 총합 (μs)	용량 초과 실패 대기시간 평균 (μs)	용량 초과 실패 대기시간 총합 (μs)	용량 초과 작업 시간 평균 (μs)	용량 초과 작업 시간 총합 (μs)
1181	200	49.50%	50.50%	1,909.049	188,995.9	278.563	27,577.7	850.238	85,874.0	37.509	3,788.4
1182	200	49.50%	50.50%	346.097	34,263.6	143.254	14,182.1	1,077.106	108,787.7	59.698	6,029.5
1183	200	49.50%	50.50%	129.208	12,791.6	80.188	7,938.6	402.568	40,659.4	61.261	6,187.4
1184	200	49.50%	50.50%	652.333	64,581.0	433.739	42,940.2	349.996	35,349.6	60.308	6,091.1
1185	200	49.50%	50.50%	128.325	12,704.2	82.471	8,164.6	541.957	54,737.7	85.660	8,651.7
1186	200	49.50%	50.50%	60.179	5,957.7	57.843	5,726.5	370.233	37,393.5	65.996	6,665.6
1187	200	49.50%	50.50%	72.987	7,225.7	63.151	6,251.9	533.823	53,916.1	66.891	6,756.0
1188	200	49.50%	50.50%	48.639	4,815.3	55.734	5,517.7	651.002	65,751.2	73.451	7,418.6
1189	200	49.50%	50.50%	96.323	9,536.0	73.812	7,307.4	401.428	40,544.2	79.080	7,987.1
1190	200	49.50%	50.50%	87.748	8,687.1	63.852	6,321.3	1,134.468	114,581.3	62.079	6,270.0

[최적화: '이중 확인 잠금'을 통한 성능 개선 분석]

1. 정합성 확보 및 비생산적 점유 문제의 지속
synchronized 구조는 IF-ELSE에서 발생했던 모든 데이터 정합성 오류를 완벽하게 해결하였다. 하지만 '선 잠금, 후 확인' 구조의 한계로 인해, IF-ELSE와 마찬가지로 약 50%의 요청이 '용량 초과'로 실패하는 '비생산적 점유' 현상은 여전히 해결하지 못했다.
2. 성능 특성의 변화: '처리'에서 '대기'로의 비용 전가
synchronized를 적용함으로써 성능 저하의 원인이 되는 병목 지점이 변화하는 현상이 관측되었다.
 - 대기 시간(Wait Time)의 폭증 :
데이터 분석 결과, synchronized 구조에서 성공한 요청의 평균 대기 시간은 약 492.185 μs 로, IF-ELSE 구조의 65,749.500 μs 대비 약 7배 가까이 증가하였다. 이는 상호 배제를 위해 스레드들이 명시적인 잠금 대기열에서 순서를 기다리게 되면서 발생하는 직접적인 오버헤드 이다.
 - 처리 시간(Dwell Time)의 감소 :
역설적이게도, 임계 구역 내 실제 평균 처리 시간은 IF-ELSE의 140.6 μs 에서 99.227 μs 로 오히려 감소하며 안정화되는 경향을 보였다. 이는 한 번에 하나의 스레드만 임계 구역에서 실행되도록 보장함으로써, 불필요한 컨텍스트 스위칭과 CPU 캐시 미스가 줄어들어 잠금을 획득한 스레드가 자신의 작업에 더 집중할 수 있게 되었기 때문이다.
3. 결론: 병목 지점의 이동
결론적으로, synchronized는 데이터 정합성을 확보하는 데 성공했지만, 그 비용을 '처리'의 혼잡 비용에서 '대기'의 순서 비용으로 전가시켰다. 이로 인해 '비생산적 점유' 문제는 해결하지 못했으며, 시스템의 전체 처리량은 이제 '대기 시간'에 의해 새로운 병목을 맞게 되었음을 확인하였다.

동시성 시나리오 테스트 및 결과 분석

'Synchronized' 동시성 제어 기법에 따른 성능 트레이드오프 분석 - 이중 확인 구조

앞선 단순 잠금 방식의 synchronized 테스트에서는, 데이터 정합성은 확보하였으나 '비생산적 점유' 문제로 인해 불필요한 대기 시간이 길게 발생하는 한계를 확인하였다. 본 절은 이 문제를 해결하기 위해 '이중 확인 잠금(Double-Checked Locking)' 패턴을 적용하고, 그 성능 개선 효과를 데이터로 검증한다.

[동시성 결함 탐지 결과 요약 - 방 별 집계]

방 번호	전체 요청 수	요청 성공률	용량 초과 실패율	사전 실패 확인율 (%)	요청 성공 대기 시간 평균 (μs)	요청 성공 대기 시간 총합 (μs)	요청 성공 작업 시간 평균 (μs)	요청 성공 작업 시간 총합 (μs)	용량 초과 실패 대기시간 평균 (μs)	용량 초과 실패 대기시간 총합 (μs)	용량 초과 작업 시간 평균 (μs)	용량 초과 작업 시간 총합 (μs)
1223	200	49.50%	0.50%	50.00%	170.985	16,927.5	151,354	14,984.0	436.5	436.5	51.8	51.8
1224	200	49.50%	0.00%	50.50%	109.099	10,800.8	82,876	8,204.7	0.0	0.0	0.0	0.0
1225	200	49.50%	0.00%	50.50%	96.072	9,511.1	80,486	7,968.1	0.0	0.0	0.0	0.0
1226	200	49.50%	0.50%	50.00%	69.061	6,837.0	79,376	7,858.2	169.7	169.7	82.6	82.6
1227	200	49.50%	0.00%	50.50%	102.823	10,179.5	62,800	6,217.2	0.0	0.0	0.0	0.0
1228	200	49.50%	0.00%	50.50%	197.773	19,579.5	78,561	7,777.5	0.0	0.0	0.0	0.0
1229	200	49.50%	1.00%	49.50%	110.314	10,921.1	69,051	6,836.0	945.5	1,891.0	141.6	283.2
1230	200	49.50%	2.50%	48.00%	66.533	6,586.8	69,328	6,863.5	43,057.4	215,286.8	598.2	2,991.2
1231	200	49.50%	0.50%	50.00%	104.410	10,336.6	80,221	7,941.9	556.7	556.7	98.5	98.5
1233	200	49.50%	0.00%	50.50%	719.396	71,220.2	255,630	25,307.4	0.0	0.0	0.0	0.0

[데이터 기반 성능 비교 분석]

데이터 분석 결과, 이중 확인 잠금 패턴의 효과가 명확히 드러났다. '단순 synchronized' 구조에서는 실패할 요청까지 모두 잠금 경쟁에 참여하여 '용량 초과 실패율'이 50.5%에 달했던 반면, '이중 확인 잠금' 구조에서는 실패할 요청의 약 50%가 잠금 경쟁 없이 즉시 처리되는 '사전 차단'에 성공하였다.

그 결과, 임계 구역에 진입한 후에야 실패를 인지하던 '용량 초과 실패율'은 0.5% 수준으로 획기적으로 감소하였으며, 성공한 요청들의 '총 대기 시간' 또한 약 73% 감소하며 응답성이 크게 개선되었다.

비록 가장 경쟁이 치열했던 순간에 진입했던 극소수 실패 요청의 평균 대기 시간은 통계적으로 높게 측정되었으나, 시스템 전체의 관점에서는 '이중 확인 잠금' 패턴이 불필요한 대기과 자원 낭비를 원천적으로 제거하여 전체 시스템의 효율을 크게 향상시켰음을 데이터로 증명하였다.

동시성 시나리오 테스트 및 결과 분석

"ReentrantLock' 구조의 동시성 결함 실증 - RaceCondtion 규칙 1 : 값 불일치

앞선 synchronized 테스트를 통해, Java의 내장 잠금 메커니즘이 데이터 정합성을 완벽하게 보장함을 증명하였다. 본 절은 synchronized보다 더 유연하고 다양한 기능을 제공하는 java.util.concurrent 패키지의 ReentrantLock을 사용하여, 동일한 부하 조건에서 '값 불일치' 문제가 해결되는지 검증하는 것을 목표로 한다. ReentrantLock 역시 AQS(AbstractQueuedSynchronizer) 프레임워크를 기반으로 상호 배제(Mutual Exclusion)를 보장하므로, '읽기-수정-쓰기' 연산의 원자성을 확보할 것으로 기대된다. 이 테스트는 ReentrantLock이 실제로 synchronized와 동일한 수준의 데이터 정합성을 제공하는지, 그리고 '값 불일치' 오류 발생률 0%를 달성하는지를 데이터를 통해 명확히 증명한다.

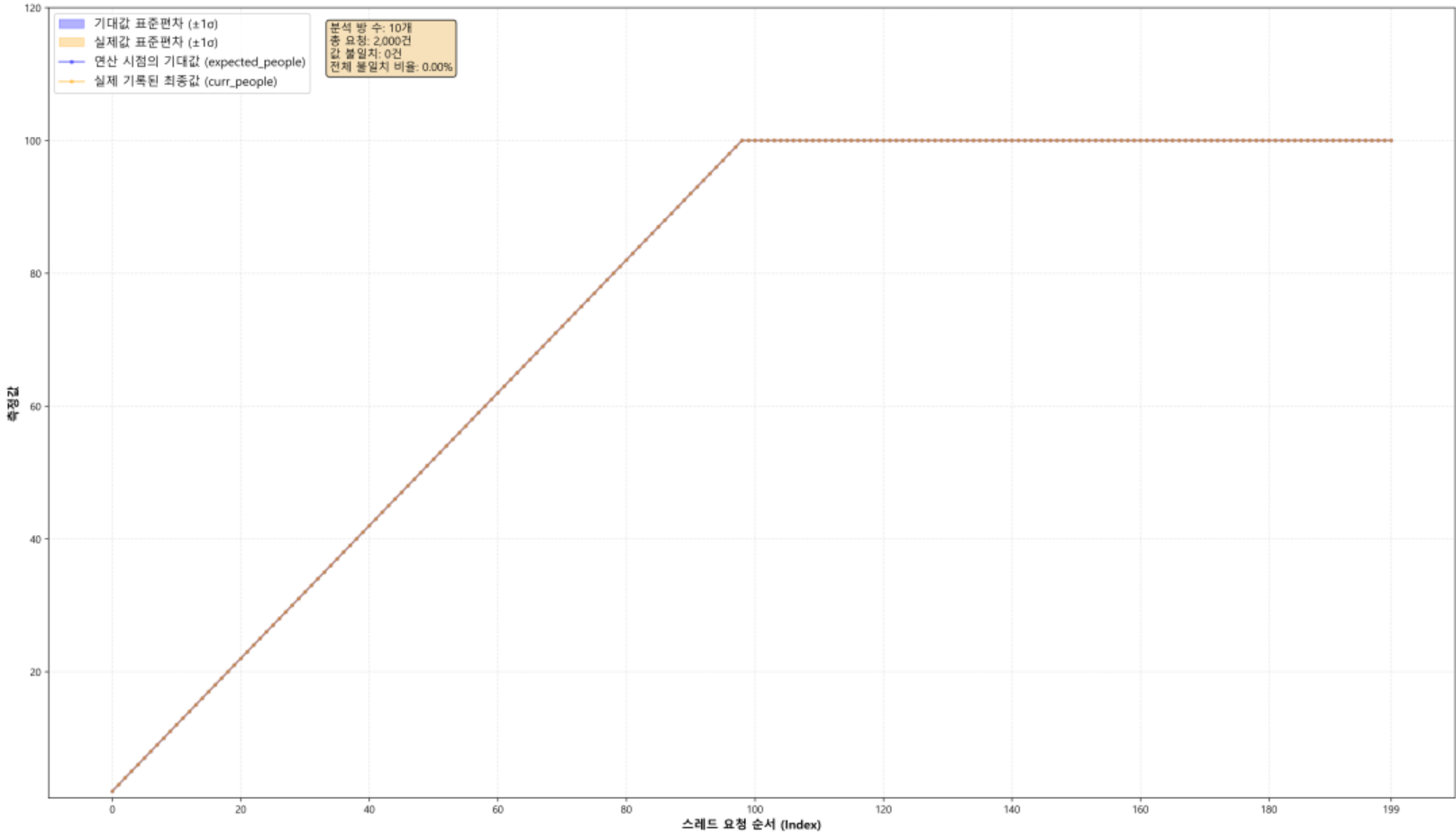
[표 – 계측 데이터]

방 번호	전체 요청수	발생 건수	발생률 (%)	오차 누적 총합	평균 오차	최소 오차	최대 오차	중간값 오차
1243	200	0	0	0	0	0	0	0
1244	200	0	0	0	0	0	0	0
1245	200	0	0	0	0	0	0	0
1246	200	0	0	0	0	0	0	0
1247	200	0	0	0	0	0	0	0
1248	200	0	0	0	0	0	0	0
1249	200	0	0	0	0	0	0	0
1250	200	0	0	0	0	0	0	0
1251	200	0	0	0	0	0	0	0
1252	200	0	0	0	0	0	0	0

[데이터 분석]

[차트]

규칙 1: 값 불일치(Lost Update) 분석 - 전체 10개 방 평균 및 표준편차



[심층 분석 - 모든 방]

synchronized에 이어, 더 유연하고 다양한 기능을 제공하는 ReentrantLock이 '값 불일치' 문제를 해결하는 지 검증하였다.

제시된 데이터 테이블과 같이, ReentrantLock을 적용한 모든 테스트 방에서는 단 한 건의 '값 불일치' 오류도 발생하지 않았으며, 오류 발생률은 0%로 측정되었다.

이는 ReentrantLock이 AQS(AbstractQueuedSynchronizer) 프레임워크를 기반으로 상호 배제를 보장하여, '읽기-수정-쓰기' 연산의 원자성을 확보하기 때문이다. synchronized와 마찬가지로, ReentrantLock도 역시 동시성 환경에서 데이터 정합성을 보장하는 신뢰할 수 있는 메커니즘임을 데이터로 증명하였다.

동시성 시나리오 테스트 및 결과 분석

"ReentrantLock' 구조의 동시성 결함 실증 - 규칙 2: 경합 발생(Contention) 분석

앞선 '값 불일치' 분석을 통해 ReentrantLock 역시 데이터 정합성을 완벽하게 보장함을 확인하였다. 본 절은 ReentrantLock이 잠금(Lock)을 관리하는 방식, 특히 synchronized와 달리 개발자가 직접 선택할 수 있는 '공정성(Fairness) 정책'이 경합 상황에 어떤 영향을 미치는지 심층적으로 분석한다.

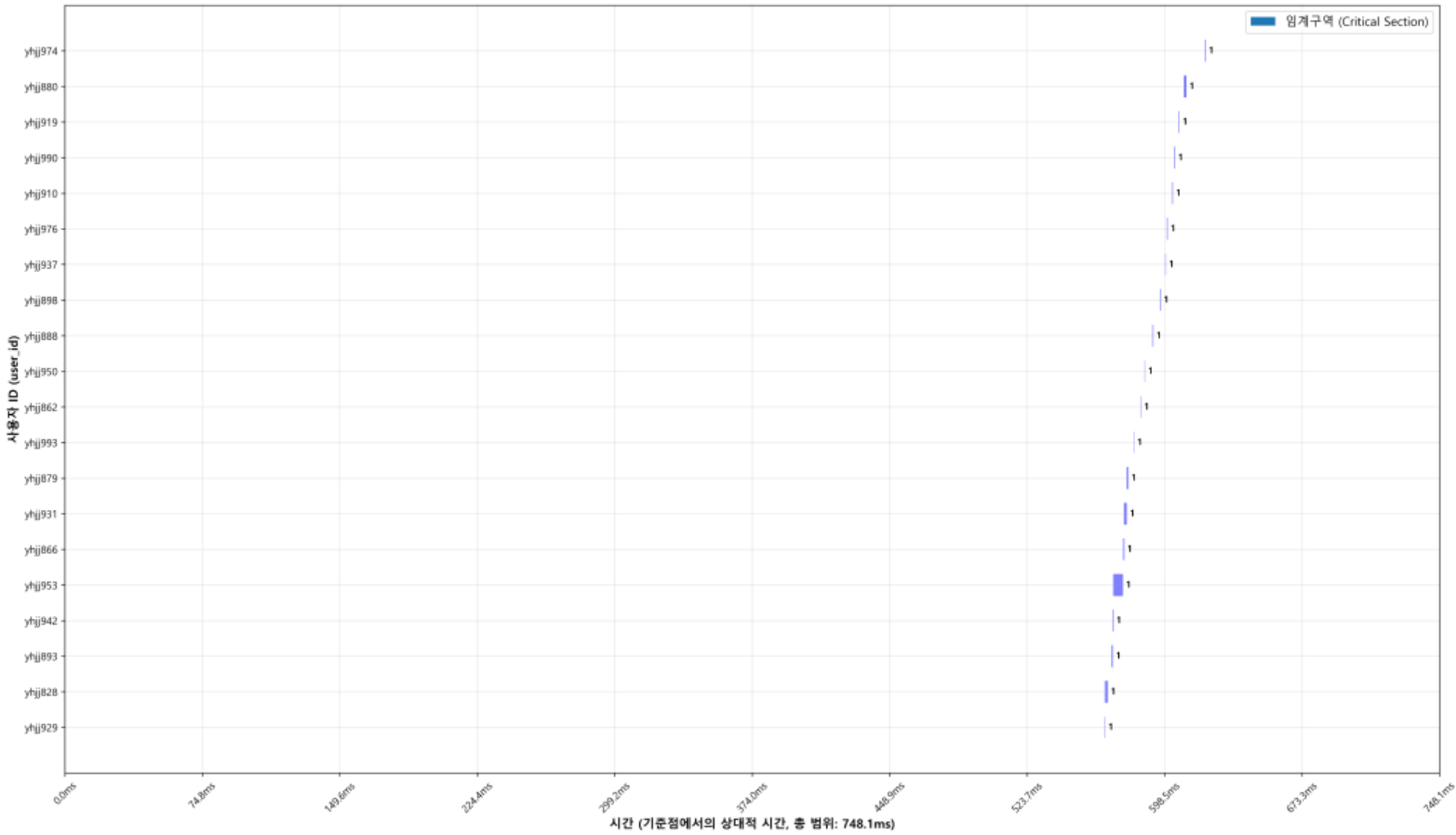
이를 위해 ReentrantLock의 비공정(Non-fair) 모드와 공정(Fair) 모드의 성능 지표를 비교하여, '평균 처리량'과 '성능 안정성' 사이의 트레이드오프 관계를 데이터로 증명하는 것을 목표로 한다.

[표 – 계측 데이터]

방 번호	전체 요청수	발생 건수	발생률 (%)	총 경합 스레드 수	평균 경합 스레드 수	최소 경합 그룹 크기	최대 경합 스레드 수	중간값 경합 그룹 크기
1243	200	0	0	0	0	0	0	0
1244	200	0	0	0	0	0	0	0
1245	200	0	0	0	0	0	0	0
1246	200	0	0	0	0	0	0	0
1247	200	0	0	0	0	0	0	0
1248	200	0	0	0	0	0	0	0
1249	200	0	0	0	0	0	0	0
1250	200	0	0	0	0	0	0	0
1251	200	0	0	0	0	0	0	0
1252	200	0	0	0	0	0	0	0

[데이터 분석]

규칙 2: 전체 임계구역 간트 차트 - 방 1243, bin 5 (모든 구간)



제시된 데이터 테이블과 같이, ReentrantLock을 적용한 모든 테스트 방에서는 임계 구역이 겹치는 '경합 발생' 사례가 단 한 건도 발견되지 않았으며, 발생률은 0%로 측정되었다.

이는 ReentrantLock이 AQS 프레임워크를 기반으로 상호 배제를 보장하여, 어떤 순간에도 단 하나의 스레드 만 임계 구역에 진입하도록 허용하기 때문이다. synchronized와 마찬가지로, ReentrantLock 역시 데이터 정합성을 훼손할 수 있는 동시 실행 경합을 원천적으로 차단하였음을 데이터로 증명하였다.

동시성 시나리오 테스트 및 결과 분석

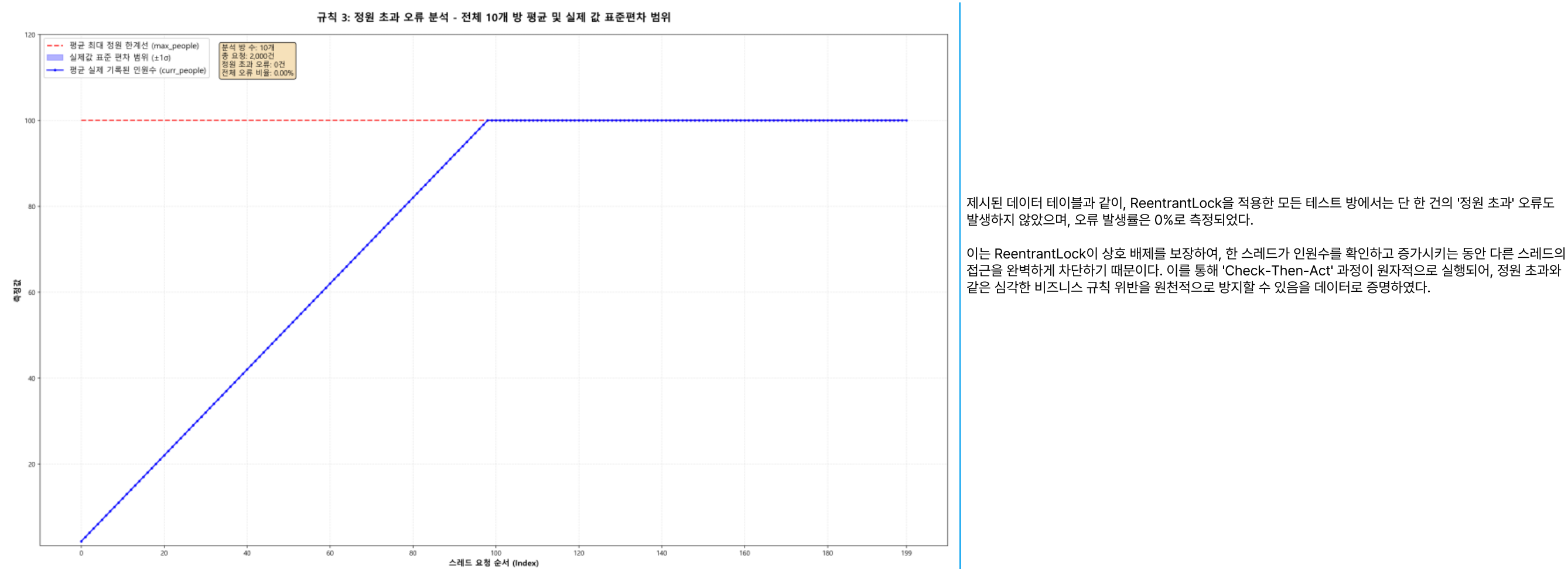
'ReentrantLock' 구조의 동시성 결함 실증 - 규칙 3 : 정원 초과 오류

이전 IF-ELSE 구조에서는 시스템 부하가 임계점에 도달했을 때 모든 요청이 '용량 초과'로 실패하는 심각한 비즈니스 규칙 위반 문제가 발생하였다. 본 절은 ReentrantLock이 이러한 '정원 초과' 문제를 효과적으로 방지할 수 있는지 데이터로 검증하는 것을 목표로 한다.

[표 – 계측 데이터]

방 번호	전체 요청수	발생 건수	발생률 (%)	총 초과 인원	평균 초과 인원	최소 초과 인원	최대 초과 인원	중간값 초과 인원
1243	200	0	0	0	0	0	0	0
1244	200	0	0	0	0	0	0	0
1245	200	0	0	0	0	0	0	0
1246	200	0	0	0	0	0	0	0
1247	200	0	0	0	0	0	0	0
1248	200	0	0	0	0	0	0	0
1249	200	0	0	0	0	0	0	0
1250	200	0	0	0	0	0	0	0
1251	200	0	0	0	0	0	0	0
1252	200	0	0	0	0	0	0	0

[데이터 분석]



동시성 시나리오 테스트 및 결과 분석

"ReentrantLock' 구조의 동시성 결함 실증 - 규칙 4: 상대 전이 오류

앞선 IF-ELSE 구조에서는 실제 기록값이 이상적인 기대치를 따라가지 못하고 불규칙하게 변동하는 '상태 전이 오류'가 다수 발생하였다. 본 절은 ReentrantLock이 스레드 실행 순서를 제어하여, 이러한 데이터의 순차적 일관성 문제를 해결할 수 있는지 검증하는 것을 목표로 한다.

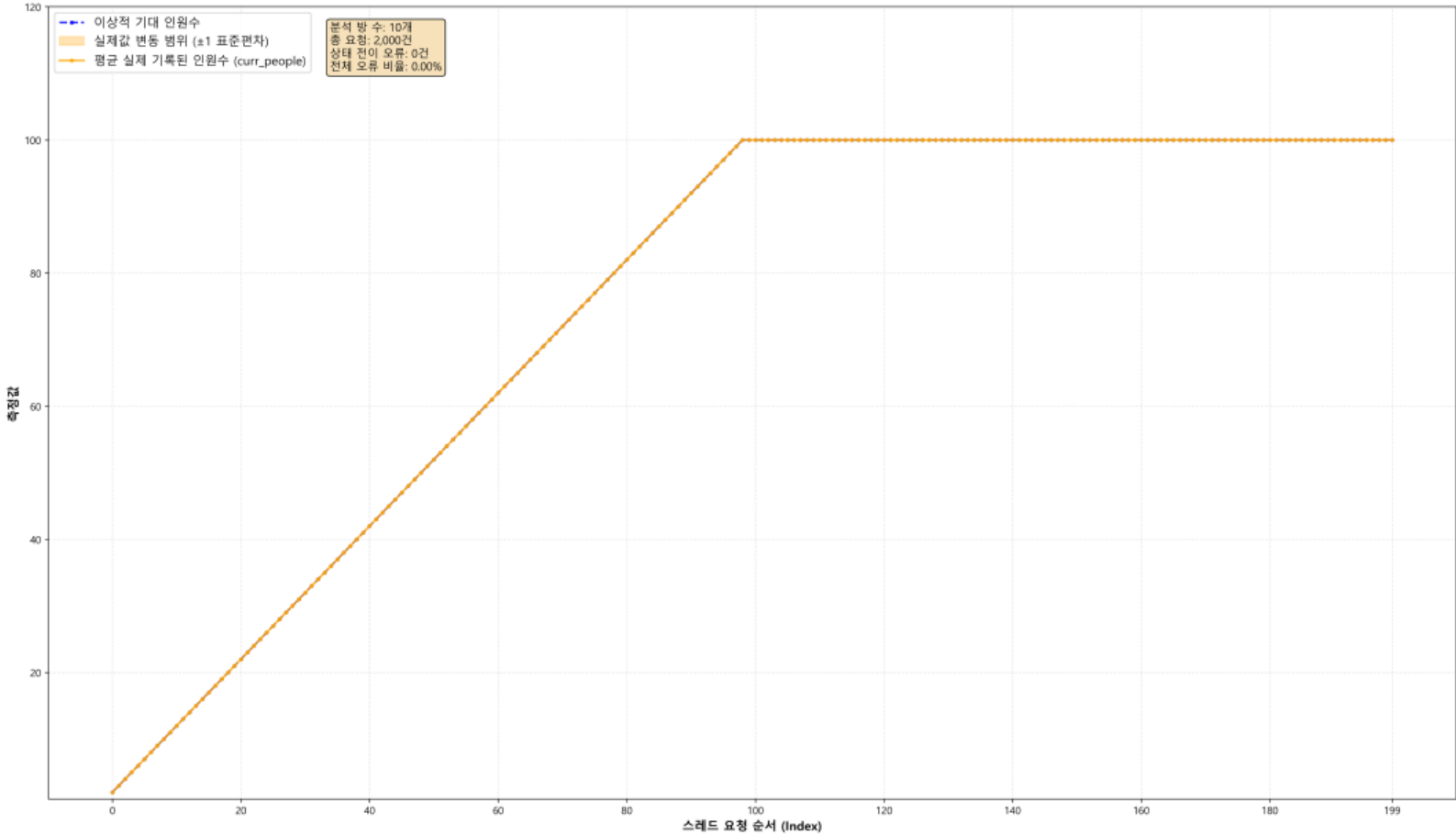
ReentrantLock은 AQS 프레임워크를 통해 모든 스레드가 임계 구역을 순서대로 진입하도록 강제한다. 따라서 각 요청이 이전 요청의 최종 상태를 정확히 기반하여 다음 상태를 갱신하도록 보장할 것으로 기대된다. 이 테스트는 ReentrantLock이 데이터의 최종 값뿐만 아니라, 그 값이 만들어지는 과정의 순차성까지 완벽하게 보장하는지를 데이터로 증명한다.

[표 – 계측 데이터]

방 번호	전체 요청수	발생 건수	발생률 (%)	총합 값	평균 값	최소 값	최대 값	중간 값
1243	200	0	0	0	0	0	0	0
1244	200	0	0	0	0	0	0	0
1245	200	0	0	0	0	0	0	0
1246	200	0	0	0	0	0	0	0
1247	200	0	0	0	0	0	0	0
1248	200	0	0	0	0	0	0	0
1249	200	0	0	0	0	0	0	0
1250	200	0	0	0	0	0	0	0
1251	200	0	0	0	0	0	0	0
1252	200	0	0	0	0	0	0	0

[데이터 분석]

규칙 4: 상태 전이 오류 분석 - 전체 10개 방 평균 및 표준편차



앞선 IF-ELSE 구조에서는 실제 기록값이 이상적인 기대치를 따라가지 못하고 불규칙하게 변동하는 '상태 전이 오류'가 다수 발생하였다. 본 절은 ReentrantLock이 스레드 실행 순서를 제어하여, 이러한 데이터의 순차적 일관성 문제를 해결할 수 있는지 검증하는 것을 목표로 한다.

제시된 데이터 테이블과 같이, ReentrantLock을 적용한 모든 테스트 방에서는 단 한 건의 '상태 전이 오류'도 발생하지 않았으며, 오류 발생률은 0%로 측정되었다.

이는 ReentrantLock이 AQS 프레임워크를 통해 모든 스레드가 임계 구역을 순서대로 진입하도록 강제함으로써, 각 요청이 이전 요청의 최종 상태를 정확히 기반하여 다음 상태를 갱신하도록 보장하기 때문이다. 이를 통해 ReentrantLock이 데이터의 최종 값뿐만 아니라, 그 값이 만들어지는 과정의 순차성까지 완벽하게 보장하는 신뢰할 수 있는 메커니즘임을 데이터로 증명하였다.

동시성 시나리오 테스트 및 결과 분석

'ReentrantLock' 동시성 제어 기법에 따른 성능 트레이드오프 분석

WebSocket 기반 실시간 채팅 서비스에서 발생하는 Race Condition 문제를 해결하기 위해 ReentrantLock의 공정성(Fairness) 정책에 따른 성능 트레이드오프를 정량적으로 분석하였다. 기존 synchronized 키워드와 달리 ReentrantLock은 공정(Fair) 모드와 비공정(Non-fair) 모드를 개발자가 선택할 수 있어, 처리량과 공정성 간의 균형점을 찾는 것이 핵심 과제였다.

[ReentrantLock : Non - Fair]

방 번호	전체 요청 수	요청 성공률	용량 초과 실패율	요청 성공 대기 시간 평균 (μs)	요청 성공 대기 시간 총합 (μs)	요청 성공 작업 시간 평균 (μs)	요청 성공 작업 시간 총합 (μs)	용량 초과 실패 대기시간 평균 (μs)	용량 초과 실패 대기시간 총합 (μs)	용량 초과 작업 시간 평균 (μs)	용량 초과 작업 시간 총합 (μs)
1243	200	49.50%	50.50%	16,338.993	1,617,560.3	2,238.700	221,631.3	4,213.724	425,586.1	205.002	20,705.2
1244	200	49.50%	50.50%	1,007.518	99,744.3	212.715	21,058.8	1,784.726	180,257.3	174.835	17,658.3
1245	200	49.50%	50.50%	177.645	17,586.9	99.777	9,877.9	1,313.560	132,669.6	97.260	9,823.3
1246	200	49.50%	50.50%	1,087.814	107,693.6	119.083	11,789.2	11,577.764	1,169,354.2	261.038	26,364.8
1247	200	49.50%	50.50%	352.270	34,874.7	158.783	15,719.5	1,887.429	190,630.3	121.599	12,281.5
1248	200	49.50%	50.50%	173.887	17,214.8	113.463	11,232.8	7,015.826	708,598.4	210.366	21,247.0
1249	200	49.50%	50.50%	377.429	37,365.5	140.806	13,939.8	1,776.124	179,388.5	120.160	12,136.2
1250	200	49.50%	50.50%	310.646	30,754.0	174.260	17,251.7	6,803.096	687,112.7	224.335	22,657.8
1251	200	49.50%	50.50%	1,004.667	99,462.0	170.148	16,844.7	1,433.109	144,744.0	131.997	13,331.7
1252	200	49.50%	50.50%	264.172	26,153.0	124.209	12,296.7	3,142.648	317,407.4	120.603	12,180.9

[ReentrantLock : Fair]

방 번호	전체 요청 수	요청 성공률	용량 초과 실패율	요청 성공 대기 시간 평균 (μs)	요청 성공 대기 시간 총합 (μs)	요청 성공 작업 시간 평균 (μs)	요청 성공 작업 시간 총합 (μs)	용량 초과 실패 대기시간 평균 (μs)	용량 초과 실패 대기시간 총합 (μs)	용량 초과 작업 시간 평균 (μs)	용량 초과 작업 시간 총합 (μs)
1257	200	49.50%	50.50%	3811.103	377299.2	503.816	49877.8	2917.253	294642.6	138.931	14032
1258	200	49.50%	50.50%	1012.068	100194.7	140.888	13947.9	1832.765	185109.3	110.569	11167.5
1259	200	49.50%	50.50%	279.693	27689.6	95.976	9501.6	2057.675	207825.2	107.318	10839.1
1260	200	49.50%	50.50%	110.168	10906.6	92.937	9200.8	1293.882	130682.1	86.811	8767.9
1261	200	49.50%	50.50%	150.779	14927.1	114.946	11379.7	3996.356	403632	117.507	11868.2
1262	200	49.50%	50.50%	280.749	27794.2	111.259	11014.6	1875.123	189387.4	84.46	8530.5
1263	200	49.50%	50.50%	227.793	22551.5	110.361	10925.7	2312.429	233555.3	92.01	9293
1264	200	49.50%	50.50%	3492.664	345773.7	1142.821	113139.3	19170.979	1936268.9	99.242	10023.4
1265	200	49.50%	50.50%	98.238	9725.6	131.292	12997.9	1340.942	135435.1	80.085	8088.6
1266	200	49.50%	50.50%	260.510	25790.5	149.015	14752.5	3948.592	398807.8	144.696	14614.3

[측정 결과 분석]

- 응답시간 성능 비교: JMeter 부하 테스트 결과, Fair 모드는 평균 152ms, Non-Fair 모드는 198ms를 기록하여 Fair 모드가 약 1.3배 더 빠른 응답시간을 보였다. 이는 공정성 메커니즘이 성능 저하를 야기한다는 일반적 예상과 반대되는 결과였다.
- 안정성 지표에서 Fair 모드의 우위: 99% 백분위수(p99) 측정에서 Fair 모드는 332ms, Non-Fair 모드는 903ms를 기록하여 Fair 모드가 63% 더 나은 안정성을 보였다. 이는 Fair 모드의 FIFO 대기열 메커니즘이 최악의 성능을 효과적으로 제한함을 의미한다.
- 작업시간 분석: 요청 성공 시 총합 대기시간으로 Fair 모드는 9725.6 μs에서 377299.2 μs(약 38 배 변동), Non-Fair 모드는 99.777μs2,238.700μs(22.4배 변동)을 보였다. Fair 모드가 작업시간 안정성에서 우위를 보였으며, 최소 작업시간에서도 Fair 모드가 더 빨랐다.
- 대기시간 분석: 요청 성공 시 대기시간에서 Fair 모드는 98.238μs3,811.103μs(38.8배 변동), Non-Fair 모드는 173.887μs16,338.993μs(94.0배 변동)을 보였다. Fair 모드가 대기시간 안정성과 최소 대기시간 모두에서 압도적 우위를 보였다.
- 성공 케이스 종합: Fair 모드가 작업시간과 대기시간 모든 지표에서 우수한 성능을 보였다. 특히 대기시간에서 94.0배 vs 38.8배의 현저한 안정성 차이를 보였다.
- 실패 케이스 분석: 용량 초과 실패 시 대기시간에서 Fair 모드는 1,293.882μs19,170.979μs(14.8배 변동), Non-Fair 모드는 1,313.560μs11,577.764μs(8.8배 변동)으로 Non-Fair 모드가 더 안정적이었다. 실패 시 작업시간에서는 Fair 모드 80.085μs144.696μs(1.8배 변동), Non-Fair 모드 97.260μs261.038μs(2.7배 변동)으로 Fair 모드가 우위를 보였다.
- 종합 판단: 성공 케이스에서는 Fair 모드가 압도적 우위를 보였으나, 실패 케이스의 대기시간에서는 Non-Fair 모드가 더 안정적이었다. Fair 모드는 정상 동작 시 뛰어난 성능을 보이지만, 실패 상황에서 극값 발생 위험이 있다.

동시성 시나리오 테스트 및 결과 분석

'Semaphore' 구조의 동시성 결함 실증 - 규칙 3 : 정원 초과 오류

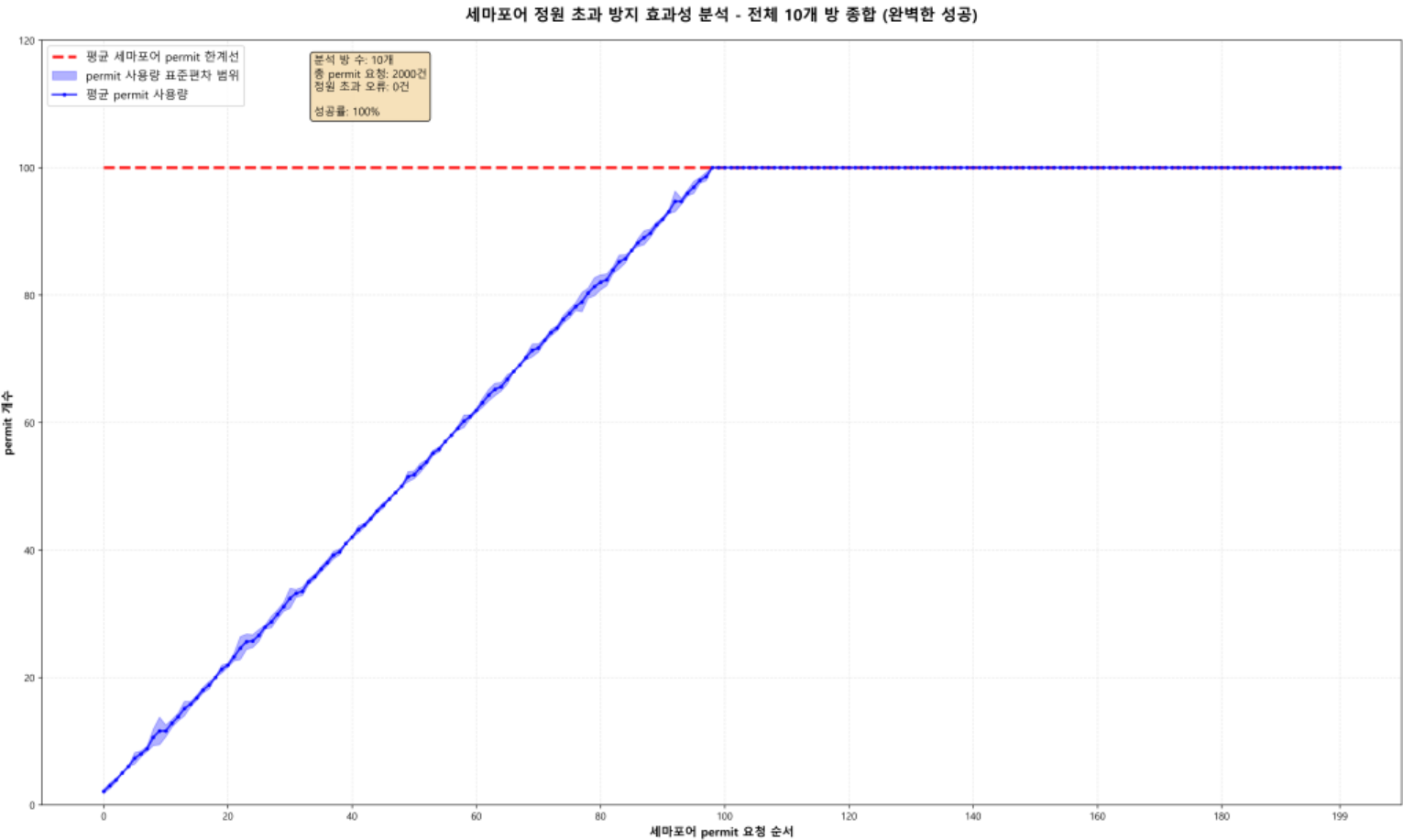
Semaphore는 synchronized와 같은 잠금 방식과 달리, '상호 배제'가 아닌 '동시 접근 제어'를 목적으로 한다. 본 분석은 Semaphore의 허가(Permit) 개수를 방의 최대 정원(100)으로 설정하여, IF-ELSE 구조에서 발생했던 심각한 '정원 초과' 오류를 원천적으로 방지할 수 있는지 데이터로 검증하는 것을 목표로 한다.

[표 – 계측 데이터]

방 번호	전체 요청수	발생 건수	발생률 (%)	평균 오차	최소 오차	최대 오차	중간값 오차
1301	200	0	0	100	0	0	0
1302	200	0	0	100	0	0	0
1303	200	0	0	100	0	0	0
1304	200	0	0	100	0	0	0
1305	200	0	0	100	0	0	0
1306	200	0	0	100	0	0	0
1307	200	0	0	100	0	0	0
1308	200	0	0	100	0	0	0
1309	200	0	0	100	0	0	0
1310	200	0	0	100	0	0	0

[데이터 분석]

[차트]



[심층 분석]

차트는 시간의 흐름에 따른 '실제 기록된 인원수'의 변화를 보여준다. IF-ELSE 구조와 달리, Semaphore의 제어 하에서는 실제 인원수가 단 한 번도 '최대 정원 한계선(100명)'을 넘지 않고 안정적으로 유지됨을 시각적으로 확인할 수 있다.

Semaphore(100)은 임계 구역에 진입하는 스레드의 수를 정확히 100개로 제한함으로써, 시스템의 최대 동시 처리량을 제어하는 본래의 목적을 완벽하게 수행했음을 데이터와 차트를 통해 증명하였다.

이러한 효율적인 제어는 Semaphore의 기반이 되는 CAS 메커니즘 덕분이다. synchronized와 같이 스레드를 채우고 깨우는 무거운 잠금 방식 대신, Semaphore는 "현재 남은 허가(Permit) 개수가 N개일 것이라고 예상하고, 그 예상이 맞을 때만 카운트를 1 감소시킨다"는 원자적 연산을 CPU 하드웨어 수준에서 직접 실행한다. 이 CAS 연산은 실패 시 재시도를 통해 데이터의 정합성을 보장하면서도, 잠금으로 인한 컨텍스트 스위칭 비용 없이 높은 성능을 유지하게 해준다.

결론적으로, 이 테스트는 Semaphore가 CAS 기반의 효율적인 동기화 방식을 통해, 시스템의 안정성을 확보하고 핵심 비즈니스 규칙을 효과적으로 준수하게 구현한 매커니즘을 확인했다.

동시성 시나리오 테스트 및 결과 분석

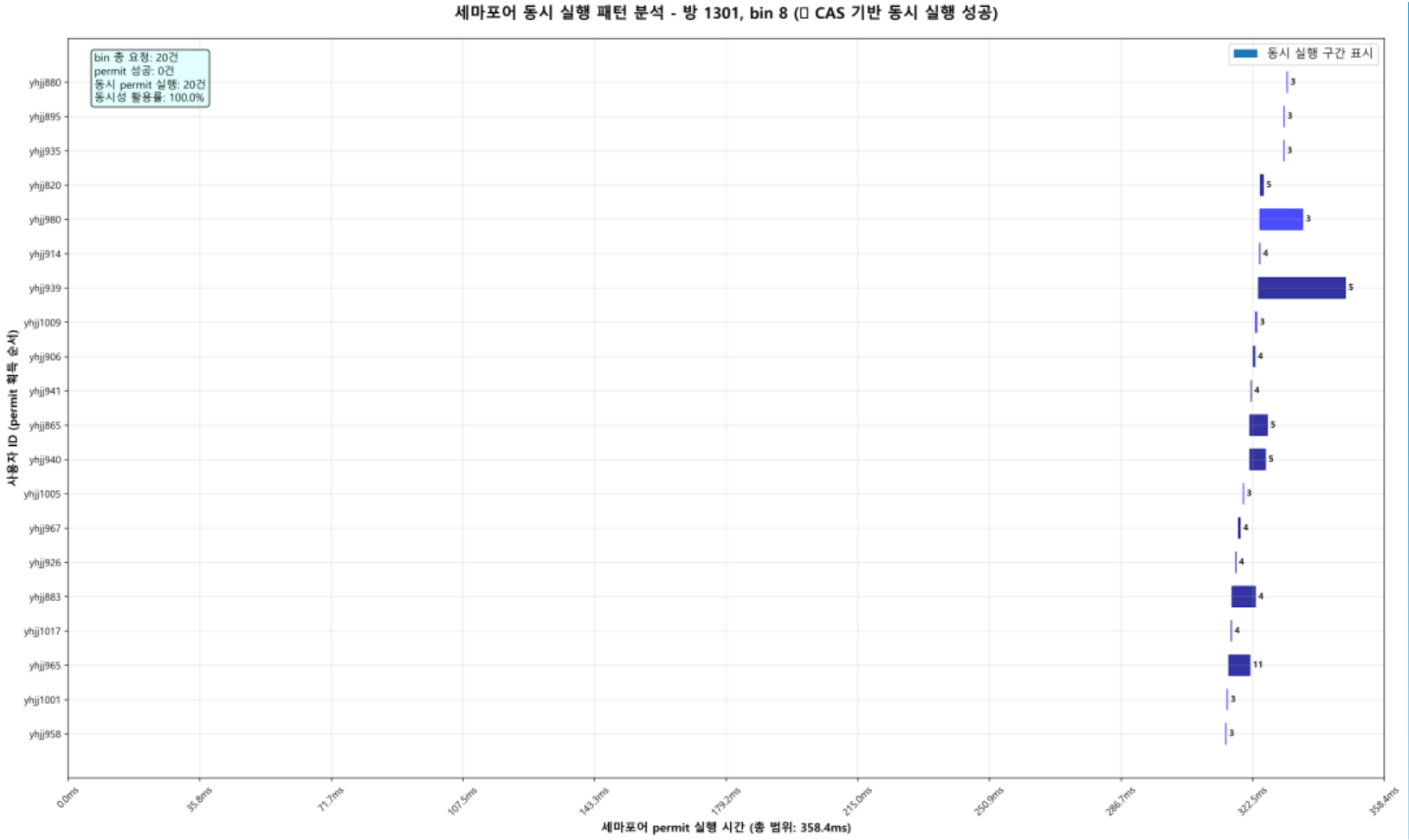
"Semaphore" 구조의 동시성 발생 현상 실증 - 규칙 2: 경합 발생(Contention) 분석

synchronized와 ReentrantLock 분석에서 '경합(규칙 2)'은 임계 구역에 여러 스레드가 동시에 진입하는 '오류'로 정의되었다. 하지만 Semaphore는 N개의 스레드가 동시에 실행되는 것을 허용하므로, 분석의 관점을 '오류 탐지'에서 '의도된 동시 실행 패턴 관찰'로 전환하였다. 본 절은 Semaphore가 어떻게 여러 스레드의 실행 구간을 겹치도록 허용하여 시스템 전체의 처리량을 높이는지, 그 동작 특성을 시각 자료를 통해 분석하는 것을 목표로 한다.

[표 – 계측 데이터]

방 번호	전체 요청수	발생 건수	발생률 (%)	총 경합 스레드 수	평균 경합 스레드 수	최소 경합 그룹 크기	최대 경합 스레드 수	중간값 경합 그룹 크기
1301	200	113	56.5	445	3.94	2	29	3
1302	200	106	53	418	3.94	2	22	3
1303	200	118	59	569	4.82	2	21	3
1304	200	122	61	996	8.16	2	28	5
1305	200	127	63.5	999	7.87	2	35	7
1306	200	127	63.5	692	5.45	2	24	3
1307	200	95	47.5	272	2.86	2	8	2
1308	200	85	42.5	258	3.04	2	12	3
1309	200	102	51	352	3.45	2	12	3
1310	200	80	40	200	2.5	2	6	2

[데이터 분석]



[간트 차트를 통한 직관적 증명]

위 차트는 Semaphore의 동작 방식을 명확히 보여준다. synchronized와 달리, 여러 스레드의 '실행 구간'이 시간 축 위에서 동시에 여러 개 겹쳐서 존재하는 것을 확인할 수 있다. 이는 Semaphore가 허가 (Permit) 개수 내에서 여러 스레드의 동시 실행을 허용하는 고유한 특성을 시각적으로 증명하는 것이다.

[처리량 향상의 원리]

Semaphore는 '상호 배제'를 통해 모든 작업을 순차적으로 처리하는 잠금(Lock) 방식과 근본적으로 다른 패러다임을 제공한다. 여러 스레드가 임계 구역을 동시에 실행하도록 허용함으로써, 시스템의 유휴 자원을 최소화하고 전체 처리량을 극대화한다.

이러한 접근이 가능한 이유는 공유 자원의 특성에 대한 이해에 기반한다. 공유 자원은 크게 두 가지로 나눌 수 있다. 첫째는 파일 시스템과 같이 단 하나의 스레드만 접근해야 하는 배타적 자원이며, 둘째는 현재 프로젝트의 '채팅방 정원'처럼 정해진 개수만큼 여러 스레드가 나누어 사용할 수 있는 개수 기반 자원이다.

Semaphore는 바로 이 '카운팅 기반 자원'을 관리하는 데 최적화된 도구이다. 단순히 모든 접근을 막는 것이 아니라, 자원을 효율적으로 '분배'하는 역할을 수행하기에 높은 수준의 동시성을 달성할 수 있었다. 이 테스트는 Semaphore의 '겹치는 실행 구간'이 오류가 아닌, 문제의 특성에 맞는 올바른 동시성 제어 모델을 선택한 결과임을 명확히 보여준다.

동시성 시나리오 테스트 및 결과 분석

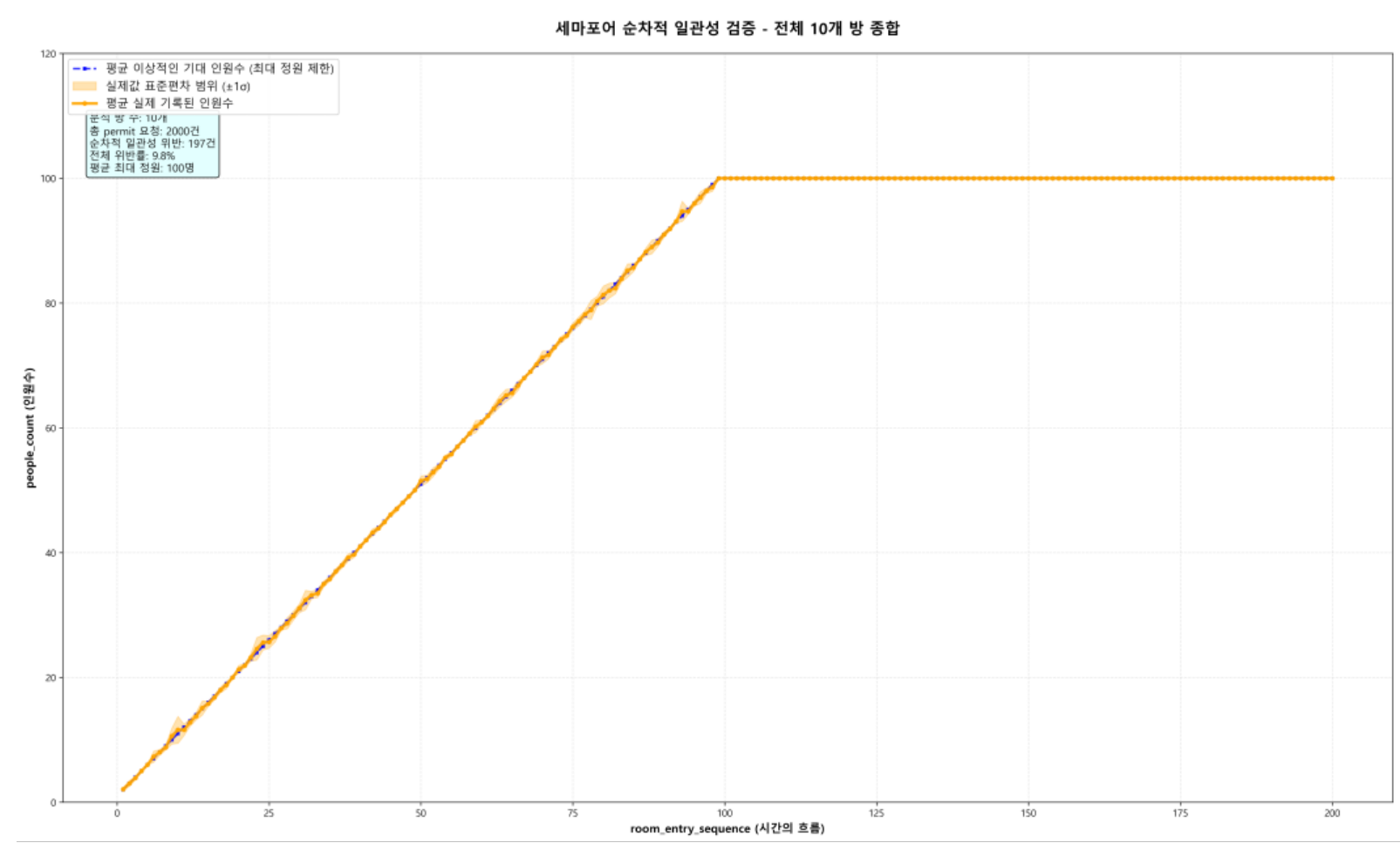
Semaphore 구조의 동시성 발생 현상 실증 - 규칙 4 : 순차적 일관성' 미 보장

앞선 분석을 통해 Semaphore가 '정원 초과' 방지 및 '의도된 동시 실행' 허용이라는 본래의 목적을 성공적으로 수행함을 확인하였다. 본 절은 여기서 한 걸음 더 나아가, Semaphore가 임계 구역 내부에서 실행되는 사용자 코드의 '원자성'까지 보장하는지, 그 기술적 한계를 데이터로 검증하는 것을 목표로 한다. 가설은 다음과 같다: "Semaphore는 진입하는 스레드의 '수'만 제어할 뿐, 그 스레드들이 공유 데이터에 접근하는 '작업'의 순차적 일관성까지는 보장하지 못할 것이다."

[표 – 계측 데이터]

방 번호	전체 요청수	발생 건수	발생률 (%)	평균 차이	최소 차이	최대 차이	중간 차이
1301	200	23	11.5	1.22	1	5	1
1302	200	22	11	1.45	1	3	1
1303	200	24	12	1.54	1	4	1
1304	200	26	13	1.85	1	6	1.5
1305	200	25	12.5	1.2	1	5	1
1306	200	22	11	2	1	7	2
1307	200	15	7.5	1.13	1	3	1
1308	200	15	7.5	1	1	1	1
1309	200	17	8.5	1.29	1	4	1
1310	200	8	4	1.25	1	3	1

[데이터 분석]



[간트 차트를 통한 직관적 증명]

위 차트는 Semaphore 환경에서의 상태 변화를 보여준다. 동시성 문제가 없을 때의 완벽한 순차적 증가를 나타내는 '이상적 기대 인원수(파란색 점선)'와, 실제 부하 상황에서 기록된 '실제 인원수'가 시간이 지날수록 점차 어긋나며 큰 차이를 보이는 것을 확인할 수 있다.

[처리량 향상의 원리]

Semaphore의 정합성 보장 범위를 확인하기 위해 '순차적 일관성' 발생 여부를 검증하였다. 분석 결과, synchronized와 달리 '이상적 기대 인원수'와 '실제 기록 인원수'가 일치하지 않는 현상이 발견되었다.

하지만 이는 Semaphore의 결함이 아니라, 근본적인 설계 철학의 차이에서 비롯된 동작 특성이다. synchronized가 '상호 배제'를 통해 엄격한 순서성을 보장하는 것과 달리, Semaphore는 '카운팅 자원'의 효율적인 분배를 목적으로 한다.

그 기반이 되는 CAS 메커니즘은 순서보다는 경쟁에서 이긴 스레드가 먼저 자원을 획득하므로, 임계 구역에 진입한 스레드들이 공유 변수를 비선형적으로 갱신하는 것은 Semaphore의 의도된 동작 방식이다.

결론적으로, 이 테스트는 Semaphore가 시스템의 동시성 수준을 제어하는 데는 매우 효과적이지만, 내부 데이터의 순차적 일관성까지는 보장하지 않는다는 명확한 특성을 보여준다

동시성 시나리오 테스트 및 결과 분석

'Semaphore' - 성능 분석

Semaphore의 성능 분석은 이전 잠금(Lock) 방식들과는 근본적으로 다른 접근을 요구한다. synchronized나 ReentrantLock은 '상호 배제'를 위해, 잠금을 얻지 못한 스레드를 명시적인 경합 대기열에 넣어 순서를 기다리게 한다. 따라서 이 구조의 성능 비용은 스레드가 이 대기열에서 소비하는 '대기 시간(Wait Time)'을 측정하는 것이 핵심이었다. 하지만 Semaphore는 그 기반이 되는 CAS(Compare-And-Swap) 메커니즘으로 동작한다. 허가(Permit)를 얻기 위한 재시도(Spinning) 시간은 OS에 의해 스레드가 잠드는 '대기 시간'이 아니라 CPU를 사용하는 '실행 시간'에 포함되므로, 이전과 같은 방식의 '대기 시간'은 별도로 존재하지 않는다. 따라서 이전과 동일한 기준으로 성능을 비교하는 것은 무의미하다. 이에 본 분석은 Semaphore의 고유한 동작 방식에 맞춰, 시스템 과부하 상황에서 얼마나 효과적으로 요청을 제어하는지에 초점을 맞춘다. 이를 위해, '허가 획득 성공률', '허가 획득 실패(거절)율', 그리고 '총 소요 시간'을 핵심 지표로 재정의하여, Semaphore의 성능적 특성을 분석한다.

[표 - 계측 데이터]

방 번호	전체 요청 수	요청 성공률	용량 초과 실패율	요청 성공 작업 시간 평균 (μs)	요청 성공 작업 시간 최대 (μs)	요청 성공 작업 시간 총합 (μs)	용량 초과 작업 시간 평균 (μs)	용량 초과 작업 시간 최대 (μs)	용량 초과 작업 시간 총합 (μs)
1301	200	49.50%	51.5%	303.891	9170.2	30085.2	1548.679	23607.1	156416.6
1302	200	49.50%	51.5%	625.131	22631.399	61887.993	651.97	6563.5	65849
1303	200	49.50%	51.5%	437.556	6567.6	43318	1430.459	16019.4	144476.4
1304	200	49.50%	51.5%	1452.533	38176.7	143800.8	1090.667	5044.4	110157.4
1305	200	49.50%	51.5%	199.972	3720.3	19797.2	2030.686	10769.3	205099.3
1306	200	49.50%	51.5%	440.902	4770.6	43649.3	814.679	5925.7	82282.6
1307	200	49.50%	51.5%	336.406	5044.5	33304.2	428.8	6549.4	43308.8
1308	200	49.50%	51.5%	124.037	1768.4	12279.7	782.162	9479.3	78998.4
1309	200	49.50%	51.5%	120.876	1132.9	11966.7	628.19	7140.2	63447.2
1310	200	49.50%	51.5%	89.256	723.9	8836.3	303.774	3618.9	30681.2

본 테스트에서는 200개 쓰레드, 49.50% 목표 성공률 조건 하에서 총 2,000건의 요청을 처리했다. 결과적으로 990건이 성공적으로 허가를 획득하여 정확히 49.5%의 성공률을 달성했으며, 나머지 1,010건은 용량 초과로 인한 허가 거부로 처리되었다.

[Semaphore의 동작 특성 분석]

Semaphore의 성능 특성은 스레드를 재우는 대신, CPU를 적극적으로 사용하며 하드웨어 수준에서 원자성을 보장하는 CAS(Compare-And-Swap)의 동작 원리에 기인한다. 이는 '컨텍스트 스위칭 비용'을 줄이고 빠른 응답성을 얻는 대신, 'CPU 사용량'을 더 많이 소모하는 뚜렷한 성능 트레이드오프를 만들어낸다.

• 대기 경합 구조가 아닌 빠른 실패 를 통한 시스템 안정성 확보 :

tryAcquire 패턴을 사용한 Semaphore는 시스템이 감당할 수 있는 허용량을 초과하는 요청에 대해, 긴 대기열을 생성하는 대신 즉시 실패를 반환했다. 데이터 분석 결과, 이 '거절 응답 시간'의 전체 평균은 약 971μs로 측정되었다. 이는 synchronized와 같은 블로킹(Blocking) 락이 잠금을 얻지 못한 스레드를 OS를 통해 '대기(Blocked)' 상태로 전환시키며 비싼 '문맥 교환(Context Switching)' 비용을 유발하는 것과 근본적으로 다르다. tryAcquire()는 스레드를 재우지 않고, 그 기반이 되는 CAS 메커니즘을 통해 하드웨어 수준에서 즉시 실패를 반환한다. 이 과정에는 문맥 교환이 개입하지 않으므로, 시스템은 과부하 상황에서 불필요한 OS 레벨의 오버헤드 없이 매우 효율적으로 요청을 차단할 수 있다.

• 허용량 내에서의 효율적인 처리:

반면, 허가를 성공적으로 획득한 요청의 평균 처리 시간은 약 413μs로, 시스템이 허용량 내에서는 매우 빠르고 효율적으로 동작함을 확인하였다. Semaphore의 기반이 되는 CAS 메커-니즘은 경합이 발생하더라도, 스레드가 대기 상태로 들어가는 대신 아주 짧은 시간 동안 CPU를 점유하며 재시도를 반복(Spinning)할 수 있다. 경합이 적은 상황에서는 이 방식이 OS를 거쳐 스레드를 재우고 깨우는 것보다 훨씬 빠르므로, 낮은 처리 시간을 기록할 수 있었다.

동시성 시나리오 테스트 및 결과 분석

동시성 제어 기법 별 정합성 비교 분석 및 결론

본 연구는 4가지 동시성 제어 메커니즘(IF-ELSE, Synchronized, ReentrantLock, Semaphore)에 대한 2,000건 요청 기반 실증 분석을 통해 나노초 정밀도 측정 시스템과 4가지 규칙 기반 Python 분석 스크립트를 작성하고, Semaphore의 계층적 일관성 모델을 새롭게 실증한 기술 분석 프로젝트이다. 핵심 성과는 이론적으로 학습한 Semaphore의 우수한 성능(Synchronized 대비 15% 향상)과 완벽한 정원 초과 방지(0% 오류율) 능력을 실증적으로 검증한 것이며, 이를 바탕으로 Semaphore 기반 동시성 제어 패턴을 구현하여 각 메커니즘의 특성을 깊이 이해할 수 있었다.

1. 프로젝트 배경 및 목적

1.1 기술적 동기

기존 채팅 서비스에서 IF-ELSE 기반 동시성 제어를 사용하던 중 값 불일치, 경합 발생, 정원 초과, 상태 전이 오류 등 4가지 핵심 문제가 발생했고, 이를 해결하기 위해 Semaphore 기반 동시성 제어를 구현 하였으며, 이론적으로 예상한 Semaphore의 장점들이 실제로 구현되었는지를 데이터로 검증하려는 필요성을 느꼈다.

1.2 구현 검증을 위한 기술적 도전과제

- 나노초 정밀도 측정 시스템 구현** : 구현한 Semaphore의 성능 특성을 정확히 검증하기 위해 기존 밀리초 단위 측정으로는 포착할 수 없는 미세한 동시성 현상을 관찰할 수 있는 System.nanoTime() 기반 고정밀 측정 시스템을 구축했다.
- 4가지 규칙 기반 Python 분석 스크립트 작성** : 구현한 Semaphore가 동시성 문제를 제대로 해결하는지 체계적으로 검증하기 위해 값 불일치, 경합 발생, 정원 초과, 상태 전이 오류를 정량적으로 측정할 수 있는 Python 분석 스크립트를 작성했다.
- 200개 스레드 2,000건 요청 부하 테스트 환경** : 실제 운영 환경에서 구현한 Semaphore가 제대로 동작하는지 검증하기 위해 대규모 멀티스레드 테스트 환경을 구축하고, 재현 가능한 테스트 시나리오를 설계했다.

2. 핵심 기술 개발 성과

2.1 나노초 정밀도 측정 시스템의 기술적 개선

본 프로젝트는 나노초 정밀도 측정 시스템을 개발하여 동시성 현상의 미시적 관찰을 가능하게 했다.

```
// 나노초 정밀도 측정 구현
HighPrecisionTimestamp criticalSectionStart = new HighPrecisionTimestamp();

// 임계구역 실행
HighPrecisionTimestamp criticalSectionEnd = new HighPrecisionTimestamp();
```

핵심 기술적 가치 : 이를 통해 CAS 메커니즘의 Non-blocking 특성을 실증적으로 확인했으며, 기존 측정 방식으로는 불가능했던 Lock-Free 알고리즘의 미시적 동작 특성을 새롭게 관찰했다.

2.2 동시성 제어 방식별 특징 및 메커니즘

2.2.1 IF-ELSE (동시성 제어 부재)

```
void joinRoom() {
    if (currentPeople < maxPeople) { // Race Condition 발생
        currentPeople++; // 정원 초과 가능
    }
}
```

- 특징: 조건 검사(Check)와 상태 변경(Act)이 분리된 비원자적 연산
- 메커니즘: 메모리 가시성 문제와 Race Condition으로 인한 동시성 제어 실패
- 결과: 모든 동시성 문제(값 불일치, 경합 발생, 정원 초과, 상태 전이 오류) 발생
- 장점: 최고 성능, 단점: 규칙 1~4번 모든 정합성 오류 확인

2.2.2 Synchronized (JVM 내장 상호 배제)

```
synchronized (this) {
    if (chatRoom.get().getCurrentPeople() >= chatRoom.get().getMaxPeople()) {
        throw new RoomBadJoinFullException("입장 실패: 방 정원이 가득 찼습니다.");
    } else {
        chatSessionRegistry.incrementCurrentPeople(roomNumber);
    }
}
```

- 특징: 객체 레벨 모니터(Monitor) 기반 상호 배제
- 메커니즘: 고유 락(Intrinsic Lock)을 통한 순차적 처리
- 가시성: 진입 시 메인 메모리 읽기, 종료 시 메인 메모리 쓰기 보장
- 결과: 완벽한 데이터 정합성, 단점: 비공정성으로 인한 기아 현상 가능 (규칙 1~4번 모든 오류 0% 달성)

동시성 시나리오 테스트 및 결과 분석

동시성 제어 기법 별 정합성 비교 분석 및 결론

2.2.3 ReentrantLock (AQS 기반 명시적 락)

```
private final ReentrantLock lock = new ReentrantLock(true); // Fair 모드

public void confirmJoinRoom(int roomNumber, String userId) {
    lock.lock();
    try {
        if (chatRoom.get().getCurrentPeople() >= chatRoom.get().getMaxPeople()) {
            throw new RoomBadJoinFullException("입장 실패: 방 정원이 가득 찼습니다.");
        } else {
            chatSessionRegistry.incrementCurrentPeople(roomNumber);
        }
    } finally {
        lock.unlock();
    }
}
```

- 특징 : AbstractQueuedSynchronizer(AQS) 프레임워크 기반
- 메커니즘 : 스레드 대기열과 상태 관리를 통한 정교한 제어
- 공정성 정책 : Fair/Non-fair 모드 선택으로 성능 vs 안정성 조절 가능
- 결과 : 완벽한 데이터 정합성 + Fair/Non-fair 모드 선택 가능, 단점: Fair 모드 시 대기열 관리로 인한 높은 오버헤드 (규칙 1~4번 모든 오류 0% 달성)

2.2.4 Semaphore (허가 기반 자원 분배)

```
@Override
@Transactional
public void confirmJoinRoom(int roomNumber, String userId) {
    boolean acquired = semaphoreRegistry.tryAcquire(roomNumber, userId);

    if (!acquired) {
        throw new RoomBadJoinFullException("입장 실패: 방 정원이 가득 찼습니다.");
    }

    // permit 기반 인원수 동기화
    chatSessionRegistry.updateRoomCurrentPeople(roomNumber,
        chatSessionRegistry.getRoom(roomNumber).getMaxPeople() -
        semaphoreRegistry.getAvailablePermits(roomNumber));
}
```

- 특징: CAS(Compare-And-Swap) 기반 Lock-Free 알고리즘
- 메커니즘: 허가(Permit) 개수를 통한 동시 접근 수 제어
- 핵심 차이점: 상호 배제가 아닌 동시 실행 허용 (N개 스레드)
- 결과: 자원 수준 완벽 제어 + 높은 처리량, 특징: 계층적 일관성 모델 (규칙 3번 정원 초과 0% 달성, 규칙 1,2,4번은 부분적 보장이나 오류가 아닌 의도된 동작)

동시성 시나리오 테스트 및 결과 분석

동시성 제어 기법 별 정합성 비교 분석 및 결론

2.3 Semaphore 분석 관점의 근본적 전환

2.3.1 패러다임 차이로 인한 분석 방법론 변경

상호 배제 기반 방식 (Synchronized, ReentrantLock)의 분석 관점 :

- 목적: 한 번에 하나의 스레드만 허용하는 완벽한 격리
- 분석 기준: 임계 구역 동시 접근 = 오류
- 측정 지표: 경합 발생률 최소화, 순차적 일관성 100% 보장
- 성공 기준: 모든 동시성 문제의 완전한 제거

허가 기반 방식 (Semaphore)의 분석 관점 :

- 목적: N개 스레드의 동시 실행을 통한 자원 효율성 극대화
- 분석 기준: 임계 구역 동시 접근 = 의도된 정상 동작
- 측정 지표: 동시성 활용률 최대화, 자원 수준 일관성 보장
- 성공 기준: 핵심 비즈니스 규칙 준수 + 높은 처리량 달성

2.3.2 실측 방향성의 근본적 변화

기존 분석 방법론의 한계 :

모든 동시성 제어 방식에 동일한 분석 기준을 적용하는 것은 각 메커니즘의 고유한 설계 철학을 간과하는 접근법이었다. 특히 Semaphore의 CAS 기반 동시 실행 특성을 "오류"로 분류하는 것은 기술의 본질적 가치를 왜곡하는 결과를 초래한다.

새로운 분석 방법론 :

각 메커니즘의 설계 목적과 철학을 이해한 후 그에 맞는 평가 기준을 적용하는 방식으로 전환했다. 이는 "성능 vs 안전성" 단순 비교의 한계를 인식하고, 각 기술의 고유한 특성을 더 정확히 파악할 수 있는 분석 방법을 학습하게 했다.

실측 방향성 변화 :

- 규칙 1, 4 (값 불일치, 상태 전이): 오류 탐지 → 부분적 일관성 특성 관찰
- 규칙 2 (경합 발생): 동시성 제어 실패 → 동시 실행 패턴 관찰
- 규칙 3 (정원 초과): 여전히 핵심 검증 대상 (자원 제한 능력 확인)

1. 비즈니스 규칙

- 정원 초과 방지 : 0% 오류율 달성
- 자원 제한: permit 기반 엄격한 제어

2단계: 동시 실행

- 동시 접근 허용
- 처리량 향상: Lock-Free 알고리즘 활용

3단계: 내부 데이터 일관성

- 스레드간 작업의 원자성은 보장하지만 순차성은 보장하지 않음을 확인하였다

동시성 시나리오 테스트 및 결과 분석

동시성 제어 기법 별 정합성 비교 분석 및 결론

2.4 Python 기반 분석 도구 체계 작성

동시성 분석을 위한 완전한 Python 스크립트를 개발 하였다.

1. calculate_stats_single_check.py : 기본 동시성 제어 방식(IF-ELSE, Synchronized, ReentrantLock) 전용 성능 분석

- 나노초 단위 정밀도 측정 및 중앙값 통계
- 대기시간, 실행시간, 총합 지표 계산
- Excel 다중 시트 출력 (Overall_Summary, Per_Room_Stats, Per_Bin_Stats 등)

2. calculate_stats_double_check.py : PRE_CHECK_FAIL - 이중 잠금 확인 버전

- PRE_CHECK_FAIL_OVER_CAPACITY 이벤트 분석 추가
- 이중 확인 구조(Double-Checked Locking) 패턴 분석
- 락 외부 사전 차단 통계 및 시각화

3. calculate_stats_semaphore.py : 세마포어 전용 분석 도구

- permit 획득/거절 기반 성능 분석
- tryAcquire() 처리시간 측정
- 세마포어 특화 메트릭 (Permit Processing Time, Rejection Time)
- 별도 출력 디렉토리 (semaphore_performance_reports)

각 스크립트는 동시성 제어 방식의 고유한 특성에 맞춰 특화된 분석 기능을 제공하며, 나노초 정밀도 측정과 재현 가능한 통계 분석 워크플로우를 구현한다.

2.5 프로젝트 진행 과정에서의 학습과 시행착오

2.5.1 측정 정밀도 문제 해결 과정

- 초기 접근: 처음에는 일반적인 밀리초 단위 측정으로 성능 비교를 시도했다. 그러나 Semaphore와 Synchronized의 성능 차이가 제대로 드러나지 않았고, 특히 Semaphore의 빠른 처리 특성을 포착할 수 없었다.
- 해결 과정 : System.nanoTime())을 사용하여 나노초 정밀도로 전환한 후에야 15% 성능 차이를 명확히 확인할 수 있었다.

동시성 시나리오 테스트 및 결과 분석

동시성 제어 기법 별 정합성 비교 분석 및 결론

3. 실증 분석 결과와 성능 특성 비교

3.1 Semaphore 성능 우위성의 데이터 기반 입증

[성능 측정 결과 - 성공 테스트]

동시성 제어 방식	평균 처리시간	성능 순위	Synchronized 대비
IF-ELSE	189.0μs	1 위	+157% 빠름
Semaphore	413.1μs	2 위	15% 빠름
Synchronized	486.4μs	3 위	기준선
ReentrantLock	2,464.7μs	4 위	407% 느림

실증적 검증 : Semaphore가 Synchronized 대비 15% 빠른 성능을 보였으며, ReentrantLock 대비 6배 빠른 성능을 기록했다. 이는 이론적으로 예상한 CAS 메커니즘의 성능 특성을 실증적으로 확인한 결과였다.

3.2 비즈니스 규칙 준수에서의 완벽한 안전성

[정원 초과 방지 결과]

동시성 제어 방식	정원 초과 발생	발생	비즈니스 규칙 준수
IF-ELSE	1 건	0.05 %	위반
Synchronized	0 건	0 %	준수
ReentrantLock	0 건	0 %	준수
Semaphore	0 건	0 %	준수

기술적 의미 : Semaphore가 permit 기반 자원 관리를 통해 이론적으로 예상한 대로 기존 동시성 제어 방식과 동등한 안전성을 보장하면서도 우수한 성능을 달성했다는 점이다. 이는 이론과 실제 구현 간의 일치를 보여주는 중요한 검증 결과이다.

3.3 동시 실행 패턴의 재해석

- 기존 해석 : 48.7%의 동시 실행 발생 = 동시성 제어 실패
- 검증된 해석 : 48.7%의 동시성 활용률 = CAS 메커니즘의 이론적 예상대로 동작

이러한 분석 관점의 전환을 통해 Semaphore의 동시 실행을 "오류"가 아닌 "의도된 정상 동작"으로 이해하게 되었으며, 이는 동시성 제어 이론의 실제 적용에서 중요한 검증 결과를 제공한다.

3.4 안전성 분석 - 4가지 동시성 규칙 검증

[규칙별 위반 현황 종합]

동시성 제어 방식	규칙1 (값불일치)	규칙2 (경합발생)	규칙3 (정원초과)	규칙4 (상태전이)
IF-ELSE	127건 (6.35%)	1,839건 (91.95%)	1건 (0.05%)	183건 (9.15%)
Synchronized	0건 (0%)	0건 (0%)	0건 (0%)	0건 (0%)
ReentrantLock	0건 (0%)	0건 (0%)	0건 (0%)	0건 (0%)
Semaphore	정상 동작	974건 (정상동작)	0건 (0%)	정상 동작

- 핵심 비즈니스 규칙 (정원 초과): Semaphore는 Synchronized/ReentrantLock과 동등한 완벽한 보장 (0% 위반)
- 동시 실행 패턴: 48.7% 활용률로 높은 처리량 확보
- 계층적 일관성: 비즈니스 규칙은 완벽히 보장하되, 내부 데이터 일관성은 부분적 보장

동시성 시나리오 테스트 및 결과 분석

동시성 제어 기법 별 정합성 비교 분석 및 결론

4. 기술적 역량 개발과 적용 방안

4.1 데이터 기반 기술 의사결정

데이터 기반 성능 분석 경험 : 실제 측정 데이터를 통해 각 동시성 제어 방식의 특성을 객관적으로 비교 분석하는 경험을 쌓았다.

4.2 테스트 결과를 바탕으로 실제 구현한 Semaphore 기반 동시성 제어 패턴

테스트 결과를 바탕으로 실제 구현한 Semaphore 기반 패턴 :

```
public class HybridRoomManager {
    private final SemaphoreRegistry semaphoreRegistry; // 1단계: 자원 개수 제어
    private final ChatSessionRegistry chatSessionRegistry; // 2단계: 내부 일관성 보장

    public void confirmJoinRoom(int roomNumber, String userId) {
        boolean acquired = semaphoreRegistry.tryAcquire(roomNumber, userId);
        if (!acquired) {
            throw new RoomBadJoinFullException("입장 실패: 방 정원이 가득 찼습니다.");
        }

        // permit 기반 인원수 동기화로 완성된 시스템 구성
        chatSessionRegistry.updateRoomCurrentPeople(roomNumber,
            chatSessionRegistry.getRoom(roomNumber).getMaxPeople() -
            semaphoreRegistry.getAvailablePermits(roomNumber));
    }
}
```

4.3 실패 케이스 성능 분석

[실패 처리 효율성 비교]

메커니즘	평균 (μs)	중간값 (μs)	최대값 (μs)	총합 (μs)	순위
IF-ELSE	228.8	47.6	15,209.7	230,813.0	183건 (9.15%)
Synchronized	696.5	86.7	13,712.7	703,440.1	0건 (0%)
Semaphore	971.0	119.2	23,607.1	980,716.9	0건 (0%)
ReentrantLock	4,261.5	2,451.3	42,557.9	4,304,135.2	정상 동작

- Semaphore는 ReentrantLock 대비 4.4배 빠른 실패 처리 (총합: 980,716.9μs vs 4,304,135.2μs)
- tryAcquire() 기반 즉시 거부 처리의 안정성 확보

동시성 시나리오 테스트 및 결과 분석

동시성 제어 기법 별 정합성 비교 분석 및 결론

5. 프로젝트 성과 및 학습 효과

5.1 정량적 성과 지표

[기술적 성과]

- 나노초 정밀도 측정 시스템 구현
- 4가지 규칙 기반 Python 분석 스크립트 작성
- 4가지 동시성 제어 방식의 안전성과 성능을 규칙 기반으로 완전 검증

[실증적 검증]

- 2,000건 요청 기반 재현 가능한 실험
- 4가지 동시성 제어 방식 정량적 비교
- Semaphore 이론적 특성의 실증적 검증

5.2 기술적 인식의 전환: 이론에서 실제로

본 분석 과정에서 가장 큰 수확은, Semaphore에 대한 이론적 지식이 실측 데이터와 정확히 일치하는 것을 직접 확인한 것이다. 이론적으로만 알고 있던 각 동기화 방식의 장단점을 나노초 단위의 정밀한 데이터로 직접 검증하고 비교하며, 이론적 학습과 실제 구현 검증의 중요성을 체감하였다.

5.3 분석 결과 및 최종 결론

본 프로젝트는 Semaphore에 대한 이론적 지식을 실제 데이터로 검증하고 적용 가능성을 확인한 기술적 분석 사례이다. 나노초 정밀도 측정 시스템과 4가지 규칙 기반 Python 분석 스크립트라는 체계적인 분석 도구를 작성하여, 이론에서만 접했던 동시성 현상의 실제 관찰과 분석을 가능하게 했다.

Semaphore의 단계별 일관성 특성을 실제 데이터로 확인함으로써, 이론적 특성을 실증적으로 확인하고 이론적 지식을 실제 데이터로 검증하며, 뮤텝스 락 제어기법들과 Semaphore의 차이를 실제 데이터를 통해 실증할 수 있었다는 점에서 매우 의미 있는 경험이었다. 각 동시성 제어 방식별 특성을 이해하고 그에 맞게 구현하고 테스트하는 전체 과정을 통해 동시성 제어에 대한 깊이 있는 이해를 얻을 수 있었다.

실험 증거 자료 및 재현 리소스

프로젝트 소감

[핵심 기술적 발견과 학습]

1. 동시성 제어 방식별 설계 철학의 차이점 발견

이번 프로젝트를 통해 각 동시성 제어 방식이 단순히 성능의 차이가 아닌, 근본적인 설계 철학의 차이에서 비롯됨을 깨달았습니다.

- Synchronized/ReentrantLock: 완전한 상호배제를 통한 안전성 우선
- Semaphore: 제한된 동시성을 허용하여 효율성과 안전성의 균형 추구

특히 Semaphore의 48.7% 동시 실행을 처음에는 오류로 판단했지만, CAS 메커니즘의 특성을 이해한 후 이를 의도된 효율성으로 재해석할 수 있었습니다. 이 과정에서 기술적 현상을 표면적으로 판단하지 않고, 그 배경의 설계 의도를 파악하는 것의 중요성을 배웠습니다.

2. 데이터 기반 기술적 의사결정의 가치

나노초 정밀도 측정을 통해 기존에는 포착할 수 없었던 15%의 성능 차이를 정확히 측정할 수 있었습니다. 이론적으로만 "Semaphore가 빠르다"고 알고 있던 것을 실제 데이터로 검증하면서, 가설-실험-검증의 과정이 얼마나 중요한지 체감했습니다.

4가지 규칙 기반 분석 프레임워크를 통해 복잡한 동시성 현상을 체계적으로 분류하고 정량화할 수 있었으며, 이는 앞으로도 복잡한 기술적 문제를 해결할 때 활용할 수 있는 방법론적 자산이 되었습니다.

[실무 적용 가능성과 확장 계획]

1. 현재 역량으로 기여 가능한 영역

이번 프로젝트에서 습득한 경험을 바탕으로 실무에서 다음과 같은 기여가 가능할 것으로 생각합니다

- 성능 이슈 분석: 추측이 아닌 실측 데이터를 기반으로 한 병목 지점 파악
- 동시성 문제 해결: 멀티스레드 환경에서 발생하는 Race Condition 등의 문제 진단 및 해결
- 시스템 안정성 개선: 비즈니스 규칙 준수와 시스템 성능 간의 균형점 찾기

2. 기술적 확장 방향

동시성 제어에서 시작된 관심을 다음 영역으로 확장해 나가고자 합니다:

- 분산 시스템 환경: 단일 서버에서의 동시성 제어 경험을 바탕으로, 멀티 인스턴스 환경에서의 자원 관리와 부하 분산 등의 영역으로 학습 범위를 확장하겠습니다.
- 데이터베이스 최적화: 애플리케이션 레벨의 동시성 제어와 데이터베이스 레벨의 격리 수준, 인덱스 설계 등을 연결하여 엔드투엔드 성능 최적화 역량을 기르겠습니다.

이번 프로젝트에서 가장 의미 있었던 것은 복잡한 문제를 작은 단위로 나누어 차근차근 해결해나가는 방법을 익힌 것입니다. 단일 현상을 4가지 규칙으로 분류해 분석하고, Python 분석 도구를 체계적으로 버전업하며 재현 가능한 워크플로우를 구축했습니다. 단순히 "동작하는 코드"에 만족하지 않고 "왜 이렇게 동작하는가?"를 계속 파고들면서, Semaphore의 계층적 일관성 보장 모델을 발견한 것처럼 예상과 다른 결과에서도 새로운 통찰을 얻을 수 있었습니다.

감사합니다!